Hybrid quantum physics-informed neural networks with a fixed quantum layer for forward and inverse problems

Gubio G. de Lima[1,*] and Erick [TODO: sobrenome][2]

[1] [TODO: afiliação (CBPF / UFSCar)]

[2] [TODO: afiliação]

We study a physics-informed neural network whose middle block is a fixed quantum circuit with no trainable parameters, on the forward and inverse problems of the viscous Burgers equation. In the forward problem, with one common protocol and noiseless simulation, the hybrid model reaches the error of a classical network of the same form with about half of the optimizer steps and fewer parameters at the short horizon, and a lower error at the long horizon. Trained with a 120-qubit superconducting processor in the loop, with one job per epoch and no error mitigation, it reaches a relative error of 5.7% in 110 epochs at the short horizon. In the inverse problem, with the quantum layer on a noisy emulator of the device, both models identify the viscosity with a median error below 3% in 500 epochs at the short horizon, with no significant difference between them; at the long horizon neither identifies it accurately (median 21% and 30%), and the classical model reconstructs the field better in all cases. Since the noiseless layer is classically computable, the simulations measure the merit of an architecture, not a quantum advantage. [TODO: Rascunho do resumo. Faltam os resultados no QPU real: problema direto em $T = 1$ e problema inverso nos dois T.]

I. INTRODUCTION

[TODO: Rascunho da introdução, a revisar.] Physics-informed neural networks (PINNs) [1, 2] represent the solution of a partial differential equation (PDE) by a neural network trained on the residual of the equation. The same construction addresses two tasks: the *forward* problem, in which the equation is known and its solution is sought, and the *inverse* problem, in which an unknown parameter of the equation is learned together with the solution from sparse data [1]. Training a PINN is delicate at small collocation budgets and long time horizons [3–5], which motivates the search for other functional forms of the network.

Quantum circuits have been proposed both as solvers of differential equations [6–8] and as layers of hybrid PINNs [9–13]. In these hybrid PINNs the circuit is trainable, which exposes the optimization to barren plateaus [14, 15] and requires gradients of the device output, for example by parameter-shift rules [16, 17], and the results come from noiseless simulation. Fixed, non-variational quantum maps have been studied as feature maps [18], reservoirs [19, 20] and convolutional filters [21], but not inside a physics-informed residual. To our knowledge, no PINN has been trained with a quantum processor in the loop. [TODO: confirmar as duas afirmações na revisão bibliográfica antes de submeter.] An obstacle is specific to this setting: the finite differences of the residual amplify the shot noise of the device.

Here we use a fixed quantum layer between two classical networks, and the same model and training machinery for both problems. All trainable parameters are classical. Since noise changes the coefficients of the layer but not its functional form, a calibrated classical surrogate of the device supplies the gradient and a noise-free interpolation across the finite-difference stencil, in a scheme analogous to physics-aware training of physical neural networks [22]. In the inverse problem the viscosity enters the residual linearly and is updated in closed form by variable projection [23].

We report four sets of results for the viscous Burgers equation, against a classical PINN of the same form: the forward problem in simulation (Sec. III A) and with the `ibm_berlin` processor in the loop (Sec. III B), and the inverse problem in simulation with noisy emulators of the device (Sec. IV A) and on real hardware (Sec. IV B, still pending). Because the noiseless layer is classically computable, the simulations measure the merit of an architecture rather than a quantum advantage [24]. Section II describes the model for both problems, Secs. III and IV give the results, and Secs. V and VI close the paper. The technical details of each of the four sets of results are in Appendices A, B, C and D, respectively.

II. HYBRID QUANTUM PINN

A. Forward and inverse problems

We consider the viscous Burgers equation on the periodic unit interval,

$$\partial_t u + u\,\partial_x u = \nu\,\partial_x^2 u, \qquad x \in [0,1),\ t \in [0,T], \tag{1}$$

with $u(x,0) = \sin(2\pi x)$. Equation (1) is the simplest PDE that combines nonlinear advection with diffusion [25]. We use $\nu = 0.01$ and two horizons. The inviscid solution breaks at $t = 1/(2\pi) \simeq 0.16$. At $T = 0.1$ the profile has steepened ($\max_x |\partial_x u| \simeq 13.6$, against 2π at $t = 0$) but no shock has formed yet. At $T = 1$ a

* gubio.gomes@estudante.ufscar.br

viscous shock sits at $x = 1/2$ and the wave has been strongly damped ($\max_x u$ drops from 0.96 at $T = 0.1$ to 0.33, with $\|u\|_{\rm rms} \simeq 0.21$), and the problem is harder.

In the *forward problem* ν is known and u is sought, with no solution data in the training. In the *inverse problem* ν is unknown, and it is recovered together with u from sparse observations

$$\mathcal{D} = \{(x_i, t_i, u_i^\star)\}_{i=1}^{N_{\rm col}}, \qquad u_i^\star = u(x_i, t_i; \nu_{\rm true}) + \varepsilon_i, \quad (2)$$

here given by the exact solution (Appendix C.1) with $\nu_{\rm true} = 0.01$ and $\varepsilon_i = 0$. This is a controlled first step towards the seismic inverse problem, in which a wave speed $c(x)$ plays the role of ν. In both problems the residual of Eq. (1) is written as

$$r = A - \nu\, B, \qquad A = \partial_t u + u\,\partial_x u, \qquad B = \partial_x^2 u, \quad (3)$$

which makes explicit that ν enters linearly.

B. Architecture

A PINN [1, 2] represents the solution by a parametrized function $u_\Theta(x, t)$. Before entering the network, the inputs are mapped to the periodic feature vector $\boldsymbol{\xi}(x, t) = (\sin 2\pi x, \cos 2\pi x, t/T)$, so that periodicity is exact. The initial condition can be imposed in two ways. With a *soft constraint* it enters the loss as a penalty, whose weight has to be balanced against the residual, a known source of training difficulties [3, 4]. With a *hard constraint* it is built into the form of the solution [26],

$$u_\Theta(x, t) = \sin(2\pi x) + \frac{t}{T}\left(\mathcal{N}_{\rm post} \circ \mathcal{M} \circ \mathcal{N}_{\rm pre}\right)\big(\boldsymbol{\xi}(x, t)\big). \quad (4)$$

We use the hard constraint everywhere except in the inverse problem at $T = 1$, where the soft constraint gave better results in training tests (Appendix C.4).

The classical pre-network $\mathcal{N}_{\rm pre}(\cdot; \boldsymbol{\phi}) : \mathbb{R}^3 \to \mathbb{R}^n$ maps the features to a latent vector $\boldsymbol{\theta} \in \mathbb{R}^n$, a middle block $\mathcal{M} : \mathbb{R}^n \to \mathbb{R}^n$ transforms it into $\boldsymbol{y}$, and the classical post-network $\mathcal{N}_{\rm post}(\cdot; \boldsymbol{\psi}) : \mathbb{R}^n \to \mathbb{R}$ produces the output (Fig. 1). The middle block is the only part that distinguishes the two models compared here, and its size n is the quantity we vary. In the *classical PINN* it is a trainable network with one hidden layer of m units,

$$\mathcal{M}_{\rm C}(\boldsymbol{\theta}) = \mathcal{N}_{\rm mid}(\boldsymbol{\theta}; \boldsymbol{\chi}). \quad (5)$$

In the *hybrid quantum PINN* (HQPINN) it is a fixed quantum circuit: the latent vector is the set of rotation angles of n qubits,

$$\mathcal{U}(\boldsymbol{\theta}) = W_{\rm brick} \bigotimes_{q=1}^{n} R_Y(\theta_q), \qquad R_Y(\theta) = e^{-i\theta Y/2}, \quad (6)$$

where $W_{\rm brick}$ is one brickwork layer of nearest-neighbour CNOT gates, and the output is the vector of local magnetizations,

$$\mathcal{M}_{\rm Q}(\boldsymbol{\theta}) = \boldsymbol{y}(\boldsymbol{\theta}), \qquad y_q(\boldsymbol{\theta}) = {\rm Tr}\big[Z_q\, \mathcal{E}_{\boldsymbol{\theta}}\big(|0\rangle\langle 0|^{\otimes n}\big)\big], \quad (7)$$

where $\mathcal{E}_{\boldsymbol{\theta}}$ is the channel actually implemented by the device. On a device y_q is estimated from S shots.

The circuit has no free parameter: all trainable parameters belong to the classical networks. The HQPINN has $21n + 61$ trainable parameters. The classical PINN has $2nm + m + n$ more in its middle block: $2n^2 + 23n + 61$ in total with $m = n$, used in the forward simulations, and $42n + 71$ with $m = 10$, used in the inverse problem and in the hardware-budget control. The two coincide at $n = 10$, with 491 parameters against 271. This is the opposite design choice to the variational hybrid PINNs studied so far [9–13], and without trainable gates the barren-plateau phenomenon of variational circuits [14, 15] does not arise.

C. Physics-informed training

At every epoch k a fresh batch of $N_{\rm col}$ collocation points (x_i, t_i) is drawn. The two problems share one form of the loss,

$$\mathcal{L}_k(\Theta) = \frac{\rho_k}{N_{\rm col}} \sum_i |r_i|^p + \frac{w_d}{N_{\rm col}} \sum_i \big|u_\Theta(x_i, t_i) - u_i^\star\big| + w_0\, \mathcal{L}_{\rm ic}, \quad (8)$$

with $r_i = r(x_i, t_i; \nu_k)$, and they differ in the terms that are switched on (Table I). The forward problem uses the squared residual alone ($p = 2$, $\rho_k = 1$, $w_d = w_0 = 0$), so that with the hard constraint Eq. (8) is the complete loss. The inverse problem uses the absolute residual ($p = 1$), which is robust to the heavy-tailed residuals of a few badly resolved points and of measurement noise, with a weight ρ_k ramped up from zero and a large fixed data weight w_d; the data term fixes the particular solution and the residual term constrains ν. At $T = 1$ it also has the soft initial-condition term $\mathcal{L}_{\rm ic}$. In all cases the derivatives in Eq. (3) are central differences on the five-point stencil $\{(x_i, t_i), (x_i \pm h, t_i), (x_i, t_i \pm \tau)\}$ with $h = 1/512$ and $\tau = 10^{-3}$, because device outputs are not differentiable with respect to the inputs.

D. Learning the viscosity

In the inverse problem ν is not updated by gradient descent. Since it enters the residual linearly [Eq. (3)], for a fixed network it can be eliminated in closed form (variable projection [23]): the minimizer of the L_1 residual is the $|B_i|$-weighted median of the ratios A_i/B_i,

$$\nu_k^\star = \arg\min_\nu \sum_{i=1}^{N_{\rm col}} |A_i - \nu B_i| = {\rm wmed}\left(\left\{\frac{A_i}{B_i}\right\};\ \{|B_i|\}\right). \quad (9)$$

Because this batch estimate fluctuates from batch to batch, the viscosity is relaxed towards it in logarithmic scale,

$$\log \nu_{k+1} = \log \nu_k + \beta_k\big(\log \nu_k^\star - \log \nu_k\big), \quad (10)$$

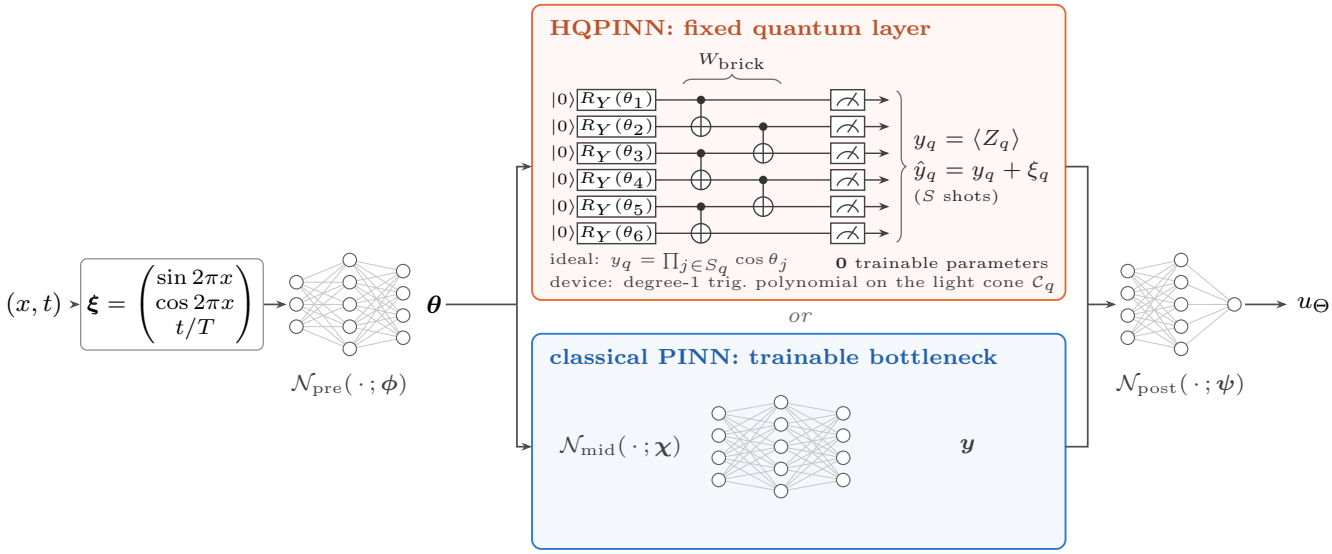


FIG. 1. Classical PINN and hybrid quantum PINN (HQPINN). Both models share the classical pre-network $\mathcal{N}_{\text{pre}}$, which maps the periodic features $\boldsymbol{\xi}$ to the latent vector $\boldsymbol{\theta}$, and the classical post-network $\mathcal{N}_{\text{post}}$, which leads to the solution u_Θ [Eq. (4)]. In the HQPINN (orange) the middle block is a *fixed* quantum circuit—one $R_Y(\theta_q)$ rotation per qubit followed by one brickwork layer of CNOTs—whose output is the vector of local expectation values $y_q = \langle Z_q \rangle$, estimated with S shots; the circuit has no trainable parameter. Six of the $n = 10$ qubits used on hardware are drawn. In the classical PINN (blue) the middle block is a trainable bottleneck $\mathcal{N}_{\text{mid}}$.

TABLE I. What changes between the two problems. In the forward problem the first value of a pair refers to the simulations, the second to the real QPU. Details in Appendices A–D.

	forward	inverse
unknown	u	u, ν
data	none	$u_i^\star$ at the points
residual norm p	2	1
ρ_k; w_d; w_0	1; 0; 0	ramp; $\gg 1$; 10 ($T = 1$)
initial condition	hard	hard; soft ($T = 1$)
N_{col}	10–100; 10	10
ν update	–	closed form
AdamW steps/epoch	1	16
budget	$\leq 2{\times}10^4$; 110	110, 500 epochs
stopping	FDM floor; budget	budget
classical hidden m	n; 10	10
evaluation	FDM, 200 points	exact, 100 points

with β_k decaying over the training (Appendix C.3). Both models use this update.

E. Structure of the quantum layer

Both the simulations and the training on a device rest on the structure of the layer, which is captured by the following statement (proof in Appendix B.1).

Proposition 1 *Let $\mathcal{C}_q \subseteq \{1, \ldots, n\}$ be the backward light cone of Z_q in the circuit (6). For any noise model in which every gate is followed by a $\boldsymbol{\theta}$-independent, trace-preserving channel acting on the qubits of that gate (readout errors included), y_q depends only on $\{\theta_j\}_{j \in \mathcal{C}_q}$ and is a multilinear trigonometric polynomial of degree one in each of them,*

$$y_q(\boldsymbol{\theta}) = \sum_{\boldsymbol{\alpha} \in \{0,c,s\}^{|\mathcal{C}_q|}} c_{\boldsymbol{\alpha}}^{(q)} \prod_{j \in \mathcal{C}_q} g_{\alpha_j}(\theta_j), \tag{11}$$

with $g_0 = 1$, $g_c = \cos$, $g_s = \sin$, and device-dependent real coefficients $c_{\boldsymbol{\alpha}}^{(q)}$. For one brickwork layer $|\mathcal{C}_q| \leq 4$, i.e., at most $3^4 = 81$ coefficients per output.

Noise therefore does not change the function class, only the coefficients. Without noise the layer reduces to products of cosines, $y_q = \prod_{j \in \mathcal{S}_q} \cos\theta_j$ with $|\mathcal{S}_q| \leq 3$ (Appendix A.4), which a classical computer evaluates efficiently: the noiseless HQPINN is a classical network with cosine-product activations. Simulations of it measure the merit of this architecture, not a quantum advantage [24].

F. Training with a device in the loop

Training on a quantum processor raises two problems: the device returns $\hat{\boldsymbol{y}}$ but not its gradient, and the finite differences of the residual amplify shot noise by a factor of order h^{-2}. Both are solved by Proposition 1. A classical surrogate $\hat{f}$ fitted to measurements describes the device, in a scheme analogous to the physics-aware training of physical neural networks [22]. One job calibrates $\hat{f}$ at random angles. At each epoch, one job measures the circuit only at the N_{col} stencil centres $\boldsymbol{\theta}_c$, and every node

$\boldsymbol{\theta}$ of the stencil centred at c receives

$$\tilde{\boldsymbol{y}}(\boldsymbol{\theta}) = \hat{f}(\boldsymbol{\theta}) + \mathrm{sg}\big[\hat{\boldsymbol{y}}^{\mathrm{QPU}}(\boldsymbol{\theta}_c) - \hat{f}(\boldsymbol{\theta}_c)\big], \tag{12}$$

where sg stops the gradient (straight-through estimator [27]). The forward pass thus uses the measured value, the gradient comes from $\hat{f}$, and the shot noise is not amplified across the stencil. In the inverse problem the measured value also enters the data term. A final job evaluates the trained network on the device. A run of K epochs costs $K+2$ jobs, independently of the number of trainable parameters (Appendices B and D). The same loop runs with a noisy emulator of the device in place of the processor.

III. RESULTS: FORWARD PROBLEM

Accuracy is measured on $u(x,T)$ against a finite-difference reference, by the mean squared error (MSE) and the relative error ϵ_{L_2} (Appendix A.1).

A. Simulation

We compare the classical PINN and the noiseless HQPINN under one common protocol, with the same optimizer, hyperparameters and step budget (Appendix A.3). Training stops when the residual on a fixed validation set reaches the *target*: the residual of the reference solution itself on those points, below which a network would be fitting the discretization error of the reference. For a given seed the two models share the validation set and therefore the target. Every configuration is repeated with 50 seeds. We report the median and interquartile range of the MSE and the mean and standard deviation of the number of steps, over all runs. A run that never reaches the target counts at the step budget, so the mean number of steps is a lower bound of the steps needed.

a. More qubits or neurons. Figure 2 shows the effect of the width n. At $T = 0.1$ with $N_{\mathrm{col}} = 100$ both models reach the same error, a median MSE of $\simeq 7\times10^{-6}$ ($\epsilon_{L_2} \simeq 0.4\%$) for $n \geq 5$, a level set by the reference solution [Fig. 2(a)]. What differs is the cost of getting there. The HQPINN uses about half of the optimizer steps, a factor between 1.95 and 2.16 for $n \geq 7$, for example 7.6×10^3 against 1.52×10^4 at $n = 10$ [Fig. 2(b)]. It also needs fewer parameters: an MSE of 8×10^{-6} takes $n = 3$ (124 parameters) against $n = 5$ (226) [Fig. 2(c)]. And it is more reliable: for $n \geq 5$, 98% of its runs reach the target, against 55% of the classical ones. With $N_{\mathrm{col}} = 10$ its median MSE is lower at every width, by a factor between 1.2 and 4.5, and an MSE of 3×10^{-5} takes $n = 2$ (103 parameters) against $n = 5$ (226).

At $T = 1$ the advantage extends to the error itself [Figs. 2(d)–(f)]. The median MSE of the HQPINN is lower at every width and for both values of N_{col}, by a factor of up to 14 for $N_{\mathrm{col}} = 100$; at $n = 10$ it is 4.8×10^{-5} ($\epsilon_{L_2} = 3.3\%$) against 3.0×10^{-4} (8.2%), and 92% of the runs reach the target against 34%. An MSE of 10^{-4} takes $n = 6$ (187 parameters), whereas the classical PINN does not reach it up to $n = 15$ (856 parameters); its best median is 1.1×10^{-4}, at $n = 9$. With $N_{\mathrm{col}} = 100$ the classical PINN uses between 1.16 and 1.6 times more steps for $n \geq 7$. With $N_{\mathrm{col}} = 10$ neither model reaches the target in most runs (at most 20% for the HQPINN and 12% for the classical PINN), so both stay close to the budget and need about the same number of steps [Fig. 2(e)].

b. More collocation points. Figure 3 shows the effect of N_{col}. At $T = 0.1$ the median MSE of the HQPINN stays below 10^{-5} from $N_{\mathrm{col}} = 30$ on, for every width [Fig. 3(a)]. The number of steps depends much more on n than on N_{col}: at $n = 10$ it goes from 1.02×10^4 at $N_{\mathrm{col}} = 10$ to 7.6×10^3 at $N_{\mathrm{col}} = 100$. At $T = 1$ the number of points matters more. For $n = 10$, going from $N_{\mathrm{col}} = 10$ to 100 lowers the median MSE from 1.3×10^{-3} to 4.8×10^{-5}, raises the fraction of runs that reach the target from 18% to 92% and lowers the mean number of steps from 1.85×10^4 to 1.33×10^4 [Fig. 3(c)]. The three-qubit model stays about an order of magnitude higher and at most 4% of its runs reach the target, which identifies the width as the limiting factor at this horizon.

The classical curves compare the two models at every width and N_{col}. At $T = 0.1$ the classical PINN uses more steps in all 50 configurations, by a factor of 1.1 to 1.7 for $n \leq 5$ and of 1.5 to 2.1 for $n \geq 7$, and it reaches the target less often in all of them; at $n = 10$, for example, in 52–64% of its runs, against at least 94% for the HQPINN from $N_{\mathrm{col}} = 20$ on [Fig. 3(b)]. The errors are close: from $N_{\mathrm{col}} = 20$ on and for $n \geq 5$ the classical median is between 0.8 and 1.5 times that of the HQPINN. Only at $n = 3$ is it clearly larger, by a factor of 1.1 to 2.8 [Fig. 3(a)].

At $T = 1$ the median MSE of the classical PINN is larger in all 50 configurations, by a factor of 1.7 to 6.2 for $n \leq 10$ and of 4.7 to 18 for $n = 15$ [Fig. 3(c)]. Among these five widths its best median is 2.1×10^{-4} ($n = 10$, $N_{\mathrm{col}} = 80$, 491 parameters); the HQPINN goes below that value with $n = 7$ at $N_{\mathrm{col}} = 30$ (208 parameters). The saving in steps grows with n and N_{col} [Fig. 3(d)]. There is none at $n = 3$, where at most 4% of the runs of either model reach the target, and it reaches a factor of 1.33 at $n = 10$ and 1.61 at $n = 15$. The HQPINN reaches the target at least as often in 49 of the 50 configurations; the exception is $n = 10$, $N_{\mathrm{col}} = 20$ (22% against 24%).

B. Real quantum processor

We trained the HQPINN with `ibm_berlin` in the loop (Sec. II F) for one specific case per horizon: $n = 10$ qubits on 8 chains in parallel (80 of the 120 qubits), one job per epoch, 110 epochs, 1024 shots and no error mitigation (Appendix B). This budget is two orders of magnitude

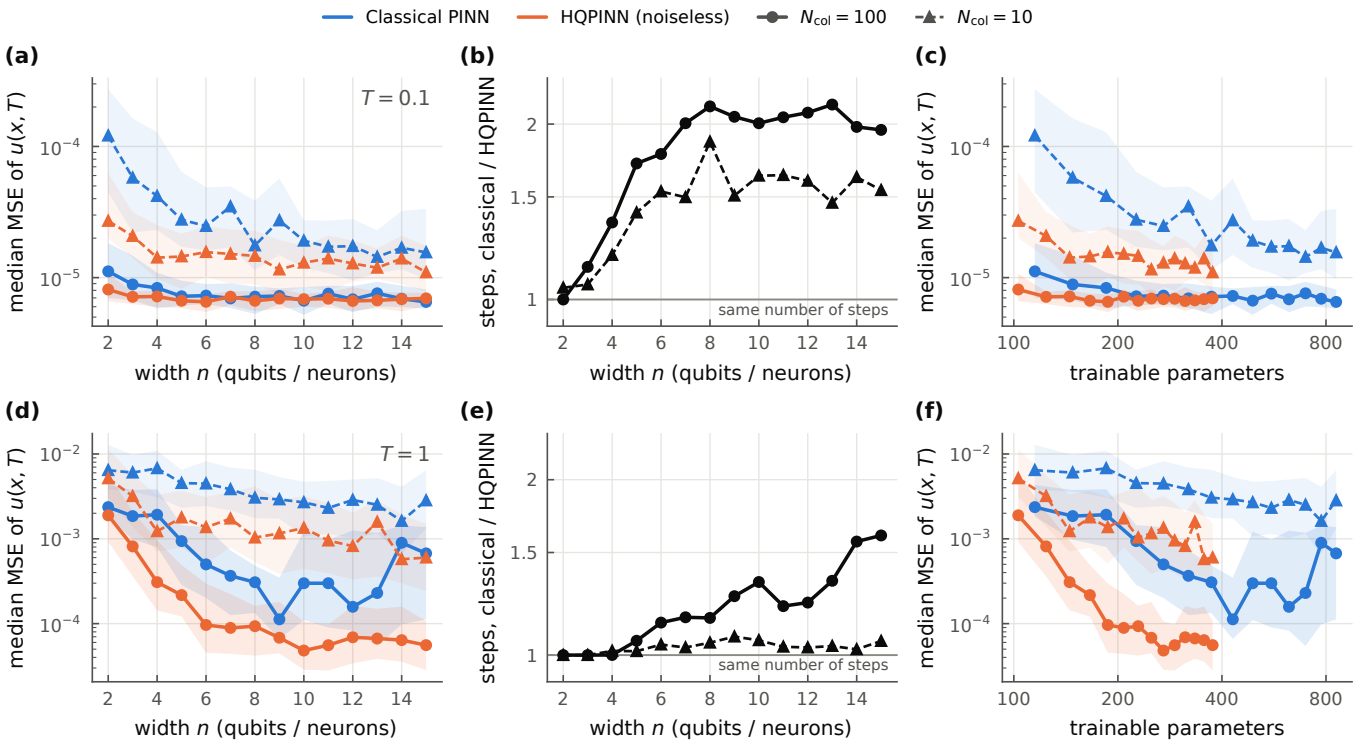


FIG. 2. Forward problem, more qubits or neurons: classical PINN (blue) and noiseless HQPINN (orange) with the same setup, for $T = 0.1$ (top) and $T = 1$ (bottom), with $N_{col} = 100$ (circles, solid) and $N_{col} = 10$ (triangles, dashed); 50 seeds per point. (a,d) Median MSE of $u(x, T)$ versus width n; bands, interquartile range. (b,e) Mean number of optimizer steps of the classical PINN divided by that of the HQPINN, over all runs, versus n (logarithmic scale); above 1 the HQPINN needs fewer steps. Runs that miss the target count at the step budget. (c,f) Median MSE versus the number of trainable parameters.

below that of the simulations, so the runs do not reach the target; here we compare the error after the same number of epochs. Each result is a single run.

a. $T = 0.1$. The network starts at $\epsilon_{L_2} = 29.4\%$ and ends at MSE $= 1.45 \times 10^{-3}$ ($\epsilon_{L_2} = 5.7\%$) when evaluated with the circuit running on the device, a 27-fold reduction of the MSE, and it follows the steep front at $x \simeq 0.5$ [Fig. 4(a)]. The classical PINN trained with the same protocol, seed and width has almost twice the parameters (491 against 271) and ends at 31.2%, with the peak and the trough displaced. Over all 50 classical runs of the control ($n = 1, \ldots, 10$, seeds 0–4) the median is 24.7%, and only one run ends below the hardware MSE [Fig. 4(b)]. The classical schedule is the one selected for the HQPINN; the best of ten schedules tested for the classical PINN at $n = 10$ gives 11.4% (6.2–19.5%), against 9.1% (3.9–15.1%) for the noiseless HQPINN with its own schedule. The hardware run lies within the spread of the noiseless model: after epoch 30 its learning curve is at or below the noiseless median in 88% of the epochs, and the same weights evaluated with the ideal circuit give 6.0%, so the network is not tied to the noise of the chip. Within the statistics of one run, device noise does not appear to be the limiting factor at this budget.

b. $T = 1$. [**TODO:** Resultado do QPU real em $T = 1$.]

IV. RESULTS: INVERSE PROBLEM

Accuracy is measured by the relative error of the learned viscosity, $\epsilon_\nu = |\hat{\nu} - \nu_{true}|/\nu_{true}$, and by the relative error ϵ_{L_2} of $u(x, T)$ against the exact solution (Appendix C.1).

A. Simulation

Both models are trained with the same algorithm (Algorithm 3 in Appendix C), with hyperparameters selected independently for each model under the same protocol. We use $\nu_{true} = 0.01$, $T \in \{0.1, 1\}$, $N_{col} = 10$ points per epoch, $n \in \{3, 7, 10\}$ and seeds 0–10, i.e., 33 runs per model and horizon. Runs with the same size and seed are paired: both models start from the same ν_0 and see the same sequence of collocation points. At $T = 1$ both models use the adjusted training of Appendix C.4. The quantum layer is evaluated on a noisy emulator of the device with $S = 1024$ shots, in the loop of Sec. II F: `FakeAachen` at the budget of $K = 500$ epochs and `FakeBerlin` at $K = 110$. The classical model runs without noise. Comparisons between the models use the two-sided paired Wilcoxon signed-rank test over the 33 pairs. Because the emulator changes together with the budget, the change

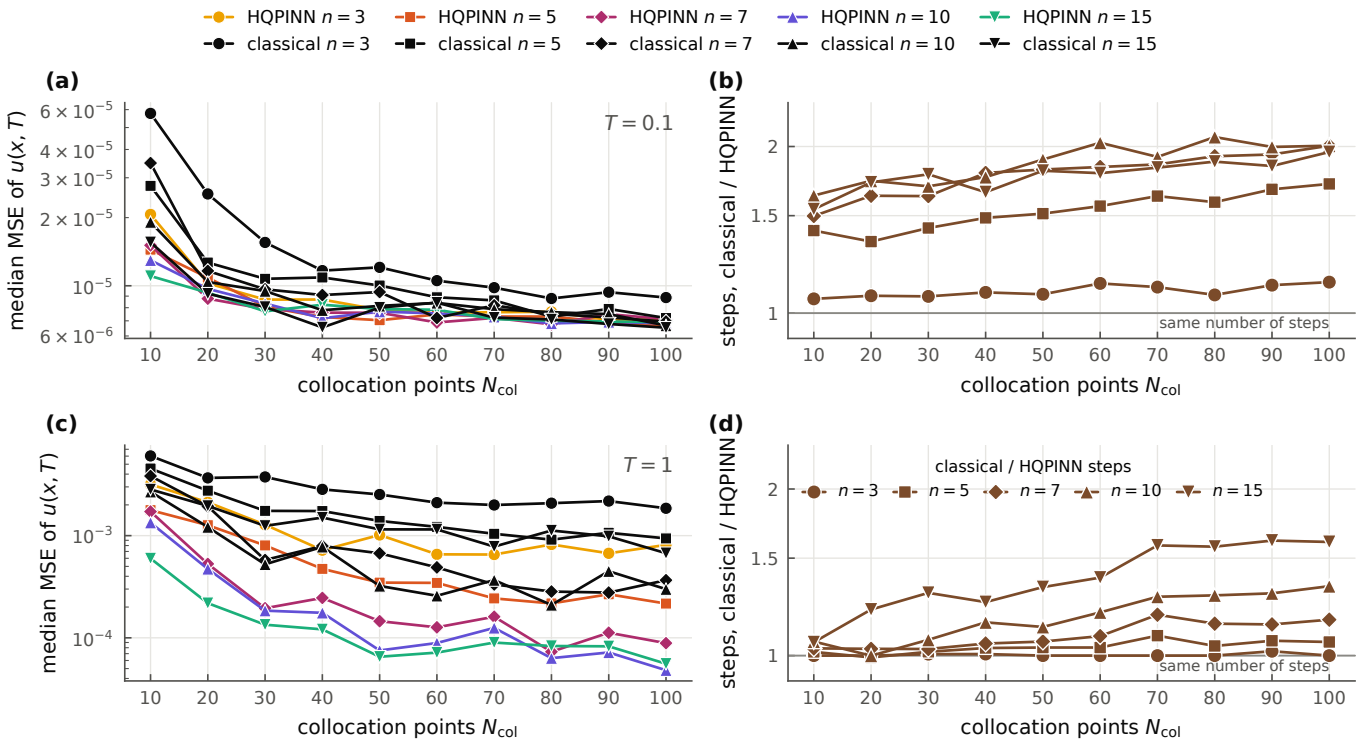


FIG. 3. Forward problem, more collocation points: noiseless HQPINN (colours, $N_{\mathrm{col}} = 10$ to 100) and classical PINN with the same setup (black, same range), with one marker per width $n = 3, 5, 7, 10, 15$, at $T = 0.1$ (top) and $T = 1$ (bottom); 50 seeds per point. (a,c) Median MSE of $u(x, T)$. (b,d) Mean number of optimizer steps of the classical PINN divided by that of the HQPINN, over all runs, one curve per width, all in one colour and told apart by the marker (logarithmic scale); above 1 the HQPINN needs fewer steps. Runs that miss the target count at the step budget.

of the HQPINN between 110 and 500 epochs mixes both effects; the classical model has no device, so its change is the effect of the epochs alone.

a. Identification of the viscosity. Table II summarizes the results over all runs. At $T = 0.1$ both models identify ν well and cannot be told apart: in 500 epochs the median ϵ_ν is 2.8% for the HQPINN and 2.3% for the classical PINN ($p = 0.76$; the HQPINN has the smaller error in 16 of the 33 pairs). The median error of both stays below 5% from about epoch 400 on. The epochs help: from 110 to 500 the classical median goes from 7.3% to 2.3% (better in 28 of the 33 runs, $p < 10^{-5}$), and that of the HQPINN from 6.0% to 2.8% (better in 24 of the 33 runs, $p = 0.001$). At 110 epochs the models are also tied ($p = 0.11$).

At $T = 1$ the problem is harder for both models and more epochs do not solve it. The median ϵ_ν goes from 21.5% to 20.9% for the classical PINN (better at 500 epochs in 13 of the 33 runs, $p = 0.26$) and from 30.2% to 30.5% for the HQPINN (17 of 33, $p = 0.82$), and the median of ν_k over the runs does not improve after about epoch 100. The classical PINN has the smaller median at both budgets, but the difference is significant only at 110 epochs ($p = 0.046$, against $p = 0.37$ at 500). Both models tend to underestimate ν: at 500 epochs in 26 of the 33 classical runs and 30 of the 33 HQPINN runs, with final values down to 3.7×10^{-3} and 2.5×10^{-3}. Across the sizes the medians vary widely and without a stable pattern: at 500 epochs, 41.2%, 27.8% and 12.8% for the HQPINN with $n = 3, 7, 10$, against 16.9%, 33.7% and 14.6% for the classical PINN, whereas at 110 epochs $n = 7$ is the best size of the HQPINN (Table X). With 11 seeds per size and a learned ν that still fluctuates by about 25% over the last 20 epochs of a run (against 1.4–1.7% at $T = 0.1$), we do not interpret these differences. The final ν of a single run depends on the last batches, and comparisons require many seeds.

b. Best runs at 110 epochs. Figure 5 shows the best run of each model, the one with the smallest ϵ_ν among its 33 runs. At $T = 0.1$ the best HQPINN ($n = 7$, seed 6) learns $\hat{\nu} = 0.01002$ ($\epsilon_\nu = 0.25\%$) and the best classical PINN ($n = 7$, seed 2) 0.00990 (1.0%); at $T = 1$, 0.01017 (1.7%, $n = 7$, seed 9) and 0.01016 (1.6%, $n = 3$, seed 2). In both cases the classical PINN reconstructs the field better, with $\epsilon_{L_2} = 0.009$ against 0.019 at $T = 0.1$ and 0.067 against 0.127 at $T = 1$, where the HQPINN overshoots the front. Panels (b,c) and (e,f) show how these runs train. At $T = 0.1$ both residual terms decrease by about an order of magnitude and ν_k settles on ν_{true} during the second half of the training; at $T = 1$ the residual term stops decreasing after the first epochs, and the ν_k of the HQPINN drops by about an order of magnitude

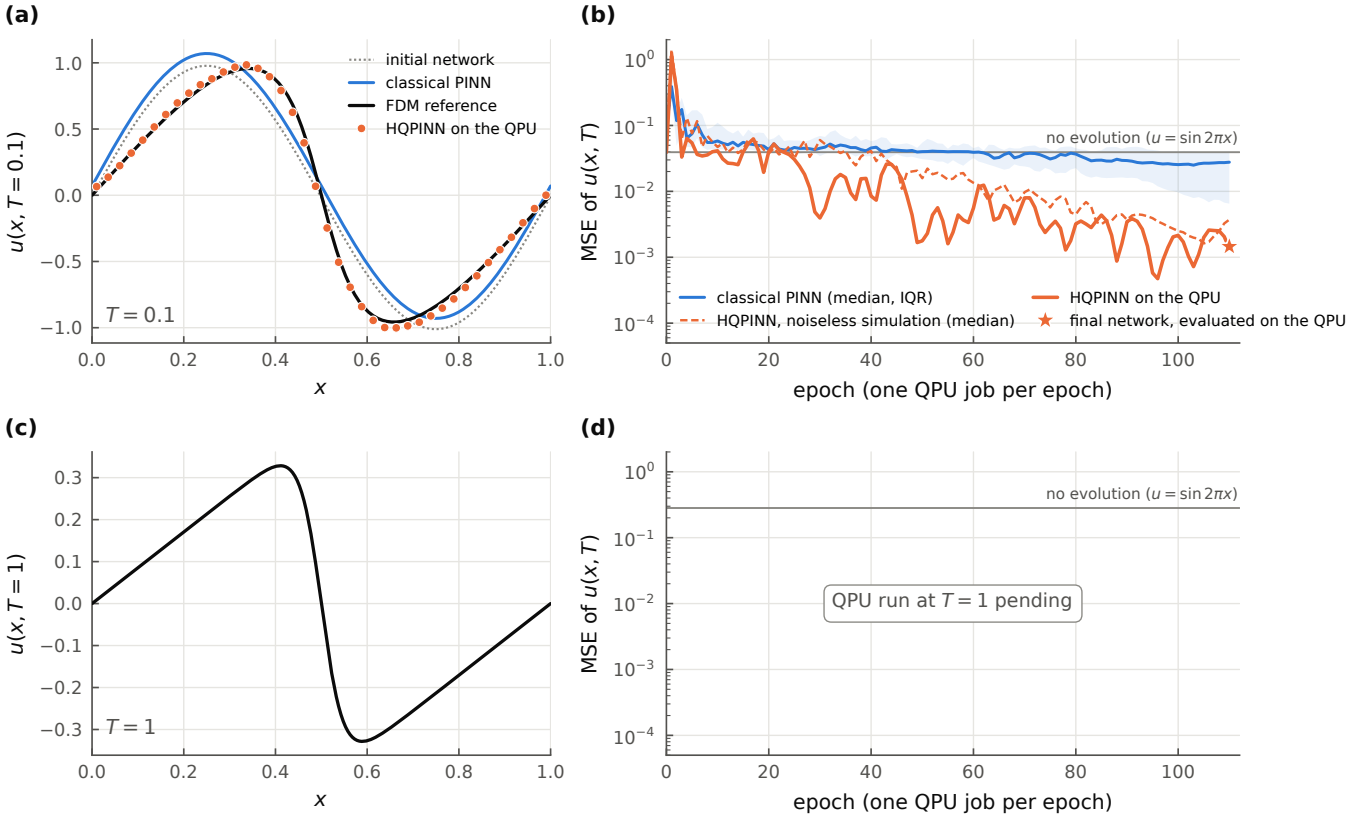


FIG. 4. Forward problem, HQPINN trained with `ibm_berlin` in the loop, one run per horizon. (a,b) $T = 0.1$: $n = 10$, $N_{\text{col}} = 10$, 110 epochs, seed 0, no error mitigation. (a) $u(x, T)$: reference, initial network, classical PINN with the same protocol, seed and width, and HQPINN evaluated on the device (every fifth point). (b) MSE of $u(x, T)$ versus epoch, from the bit-exact replay of the recorded run, through the surrogate in force at each epoch; star, final network evaluated on the device. Blue, median and interquartile range of the classical PINN with the same protocol ($n = 1, \ldots, 10$, seeds 0–4); dashed, median of the noiseless HQPINN (seeds 0–4). Grey, MSE of a network that does not evolve the initial condition. (c,d) $T = 1$. [**TODO:** Painéis (c,d) aguardam a rodada no QPU; (c) mostra só a referência.]

in single epochs before it returns.

c. Reconstruction of the field. The classical PINN, which runs without noise, reconstructs $u(x, T)$ better in all four cases (Table II): at 500 epochs, median $\epsilon_{L_2} = 0.005$ against 0.013 at $T = 0.1$ (better in 32 of the 33 pairs) and 0.044 against 0.068 at $T = 1$ (28 of 33). Unlike ν, the field keeps improving with the epochs at $T = 1$, from 0.073 to 0.044 for the classical PINN and from 0.107 to 0.068 for the HQPINN. A better field thus does not by itself give a better ν at this horizon. A likely cause is the small number of points per epoch: on a proxy of the device, more collocation points lowered the median ϵ_ν of the HQPINN at $T = 1$ markedly (Appendix C.4).

The networks of the HQPINN are trained through the noisy device, and they adapt to it. At $T = 0.1$, the same weights combined with the ideal circuit give a larger error in 22 of the 33 runs at 500 epochs, by a median factor of 1.05 (25 of 33 and 1.14 at 110 epochs); at $T = 1$ the effect disappears (17 of 33, median factor 1.00), the error being dominated by the front. This is the behaviour expected from Proposition 1: noise changes the coefficients $c_{\boldsymbol{\alpha}}^{(q)}$ but not the function class, and the post-network learns the map from the coefficients of the device.

B. Real quantum processor

[**TODO:** Ainda não há nenhuma rodada do problema inverso em hardware real: todos os resultados acima vêm de emuladores. Esta seção e a Fig. 6 aguardam os dados. Sugestão de texto a preencher: $\hat{\nu}$, ϵ_ν e ϵ_{L_2} da rodada, comparados com a mediana e a dispersão do emulador no mesmo orçamento.] The training loop, the protocol and the comparison are ready. A run on a device uses the machinery of Sec. II F, with the same training as on the emulator; at the budget of the forward run it costs 112 jobs (Appendix D). Figure 6 holds the place of the result, which will be compared with the emulator medians at the same budget.

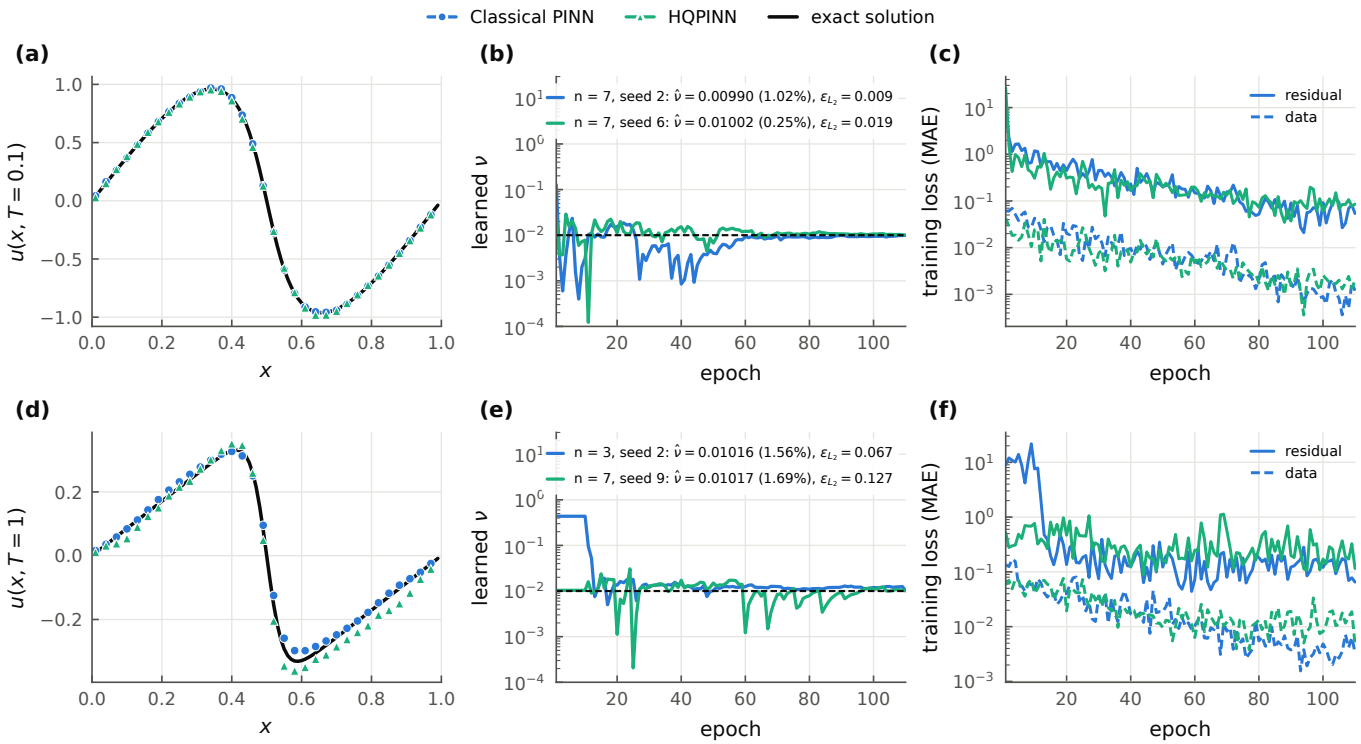


FIG. 5. Best run of each model at $K = 110$ epochs, the one with the smallest ϵ_ν among its 33 runs (blue, classical PINN; aqua, HQPINN with the circuit on `FakeBerlin`), for $T = 0.1$ (top) and $T = 1$ (bottom). (a,d) $u(x, T)$ and the exact solution. (b,e) Learned viscosity versus epoch; dashed, ν_{true}; at $T = 1$ the viscosity is frozen during the first 10 epochs. The legend gives the size and seed of each run, its final $\hat{\nu}$ and ϵ_ν, and ϵ_{L_2} of $u(x, T)$. (c,f) The two terms of the training loss per epoch, the mean absolute error of the residual (solid) and of the data (dashed).

V. FINAL CONSIDERATIONS

[**TODO:** Rascunho, a revisar depois das rodadas no QPU.] In the forward problem, the simulations show that the HQPINN reaches the same error as the classical PINN with fewer parameters and fewer optimizer steps, and at the long horizon also a lower error. This comparison is between two classical computations: the noiseless quantum layer is a fixed cosine-product map, and its better conditioning of the residual minimization is an architectural effect. The quantum processor enters as the physical device that implements this map, and the hardware experiment addresses whether such a model can be *trained* through a noisy device. At the budget tested the answer is positive: with one job per epoch, without error mitigation, on 80 qubits, the network reaches an accuracy within the spread of the noiseless model trained with the same protocol. This is possible because noise changes the coefficients of the layer but not its functional form, so a calibrated model of the device provides both the gradient and a noise-free interpolation across the finite-difference stencil.

In the inverse problem we find no advantage of the HQPINN at the budgets and with the noise tested. The two models identify ν equally well at $T = 0.1$, neither identifies it accurately at $T = 1$, and the classical PINN reconstructs the field better throughout. The same training machinery works in both problems, but the two sets of results should not be read as one comparison. The forward simulations are noiseless, long and stop at a physical target; the inverse runs are short, fixed-budget, and put the noise of the device on the HQPINN only. In addition, the hyperparameters of the inverse runs were selected at $\nu = 0.1$, $T = 0.1$ and not at the values used here. The direct counterpart of Sec. III A for the inverse problem, a noiseless HQPINN and a classical PINN under one protocol, does not exist yet: the older long-budget sweeps differ in circuit, stopping rule and number of seeds (Appendix C.8). [**TODO:** Rodar esse caso e o controle clássico com produto de cossenos [Eq. (13)] separaria o efeito da arquitetura do efeito do ruído.]

Limitations. The hardware results rest on a single run per horizon, the seed at $T = 0.1$ was the best of five on an emulator of the device, and the real-QPU results at $T = 1$ (forward) and for the inverse problem are still missing. The classical family compared here has one specific middle block, and in the forward problem the common protocol does not use separately optimized hyperparameters for each model; at the hardware budget a schedule tuned for the classical PINN narrows the gap to the noiseless HQPINN (Appendix B.5). The target and the error metric of the forward problem rely on a first-order finite-

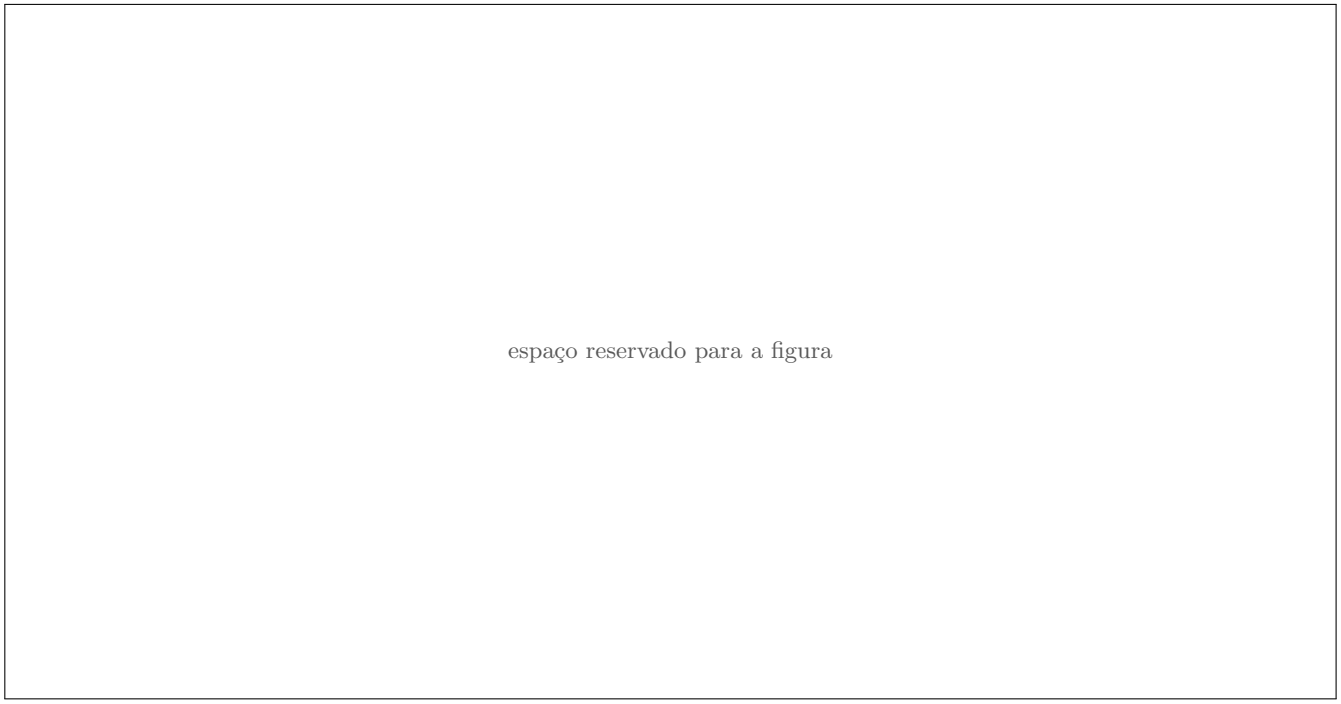


FIG. 6. Inverse problem on the real QPU. [**TODO:** A rodada ainda não existe; o script `fig7_inv_qpu.py` preenche os dados dela sozinho (`INVERSE_QPU_CASES` em `paper_common.py`).] (a,c) Learned viscosity versus epoch at $K = 110$: QPU run, and medians of the classical PINN and of the HQPINN on `FakeBerlin` over 33 runs; dashed, ν_{true}. (b,d) $u(x, T)$: QPU run and exact solution.

difference reference whose own error sets the 7×10^{-6} floor at $T = 0.1$. The emulators reproduce the calibrated errors of the device but not crosstalk or drift, and they do not fix the seed of the simulator, so rerunning the HQPINN gives slightly different numbers. In the inverse problem, the effects of the budget and of the emulator on the HQPINN were not separated.

VI. CONCLUSIONS

[**TODO:** Rascunho – revisar; incluir $T = 1$ no QPU e o problema inverso em hardware real.] A PINN with a fixed, parameter-free quantum layer needs fewer trainable parameters and fewer optimizer steps than a classical PINN of the same form to reach the same error on the viscous Burgers equation: about half of the steps at $T = 0.1$ and, at $T = 1$ and $n = 10$, an error about 6 times lower. Since the noiseless layer is classically computable, this is a property of the architecture. The model can also be trained with a superconducting quantum processor in the loop, with one job per epoch and no error mitigation: at $T = 0.1$ it reaches $\epsilon_{L_2} = 5.7\%$ in 110 epochs, in line with the noiseless simulation at the same budget. The same model and loop extend to the inverse problem, with the viscosity updated in closed form. With the layer on a noisy emulator it identifies the viscosity as well as the classical PINN at $T = 0.1$ (median error 2.8% against 2.3% in 500 epochs), and it does not improve on it at $T = 1$, where neither model is accurate.

DATA AND CODE AVAILABILITY

[**TODO:** Repositório/Zenodo.] The tables behind every figure are in `manuscript/data/`, written from the raw results in `data/` by `manuscript/scripts/build_paper_data.py`; `manuscript/scripts/make_all.py` regenerates tables and figures. `replay_training.py`, `simulate_same_budget.py` and `controls_hardware_budget.py` produce the replayed learning curves and the hardware-budget controls without any quantum job. The IBM job identifiers of all 113 hardware jobs are recorded in the run logs.

ACKNOWLEDGMENTS

[**TODO:** Agradecimentos; acesso ao IBM Quantum (instância PDE_Europe); financiamento.]

TABLE II. Inverse problem: relative error of the learned viscosity, ϵ_ν, and relative L_2 error of $u(x,T)$, over 33 paired runs per model ($\nu_{\text{true}} = 0.01$, $n \in \{3,7,10\}$, seeds 0–10). Wins: pairs in which the model has the smaller error; p: two-sided paired Wilcoxon test.

	HQPINN[b]	classical
$T = 0.1$, $K = 500$ (FakeAachen)		
ϵ_ν (%)[a]	2.8 / 3.2 / 13.4	2.3 / 3.1 / 12.5
ϵ_{L_2}[a]	0.013 / 0.014 / 0.027	0.005 / 0.006 / 0.010
wins (ϵ_ν; ϵ_{L_2})	16; 1	17; 32
p (ϵ_ν; ϵ_{L_2})	0.76; 2×10^{-9}	
$T = 1$, $K = 500$ (FakeAachen)		
ϵ_ν (%)[a]	30.5 / 30.7 / 75.0	20.9 / 26.0 / 62.8
ϵ_{L_2}[a]	0.068 / 0.077 / 0.149	0.044 / 0.044 / 0.066
wins (ϵ_ν; ϵ_{L_2})	15; 5	18; 28
p (ϵ_ν; ϵ_{L_2})	0.37; 9×10^{-7}	
$T = 0.1$, $K = 110$ (FakeBerlin)		
ϵ_ν (%)[a]	6.0 / 6.7 / 25.1	7.3 / 9.8 / 29.8
ϵ_{L_2}[a]	0.021 / 0.022 / 0.044	0.014 / 0.016 / 0.033
wins (ϵ_ν; ϵ_{L_2})	20; 8	13; 25
p (ϵ_ν; ϵ_{L_2})	0.11; 7×10^{-4}	
$T = 1$, $K = 110$ (FakeBerlin)		
ϵ_ν (%)[a]	30.2 / 32.2 / 71.1	21.5 / 23.1 / 68.5
ϵ_{L_2}[a]	0.107 / 0.112 / 0.214	0.073 / 0.071 / 0.098
wins (ϵ_ν; ϵ_{L_2})	13; 1	20; 32
p (ϵ_ν; ϵ_{L_2})	0.046; 2×10^{-9}	

[a]Median / mean / max. [b]Noisy emulator; $u(x,T)$ evaluated with the circuit run on the emulator.

Appendix A FORWARD PROBLEM: SIMULATION

A.1 Reference solution and error metrics

The reference u_{FDM} is obtained with an explicit scheme on $N_x = 512$ periodic grid points ($\Delta x = 1/512$) with time step $\Delta t = 10^{-4}$: first-order upwind differences for the advection term, $u\,\partial_x u \to u_j(u_j - u_{j-1})/\Delta x$ for $u_j \geq 0$ and $u_j(u_{j+1} - u_j)/\Delta x$ otherwise, and second-order central differences for the diffusion term. The time step satisfies the advective and diffusive stability limits ($\Delta t \max|u|/\Delta x \simeq 0.05$, $\nu \Delta t/\Delta x^2 \simeq 0.26$). Evaluation points that do not coincide with the grid are obtained by periodic linear interpolation at $t = T$. The reference is used only for evaluation: ν is known and there is no solution data in the training.

FDM floor. The stopping target of the simulations is the residual of the reference itself, $\mathcal{L}_{\text{FDM}} = \frac{1}{N}\sum_k r_k[u_{\text{FDM}}]^2$, evaluated with the same stencil (h,τ) on the same validation points, with u_{FDM} interpolated bilinearly (periodic in x, linear in t). Because the upwind scheme is only first-order accurate, $\mathcal{L}_{\text{FDM}} > 0$; a network whose residual is below it would be fitting the error of the reference. The corresponding floor of the MSE against the reference is $\simeq 7 \times 10^{-6}$ at $T = 0.1$. It is the error of the reference itself: against the exact solution of Appendix C.1, on 2000 points, u_{FDM} has an MSE of 6.9×10^{-6} at $T = 0.1$ and 4.0×10^{-6} at $T = 1$.

Metrics. MSE $= \frac{1}{P}\sum_p [u(x_p,T) - u_{\text{FDM}}(x_p,T)]^2$, MAE analogously, and $\epsilon_{L_2} = \|u - u_{\text{FDM}}\|_2 / \|u_{\text{FDM}}\|_2$, on $P = 200$ equispaced points $x_p \in [0,1]$. At $T = 0.1$, $\langle u^2_{\text{FDM}} \rangle = 0.453$, so $\epsilon_{L_2} = \sqrt{\text{MSE}/0.453}$. A model that does not evolve the initial condition, $u(x,T) = \sin 2\pi x$, has MSE $= 3.93 \times 10^{-2}$ at $T = 0.1$ and 2.82×10^{-1} at $T = 1$.

A.2 Architectures and hyperparameters

TABLE III. Network architectures (all hidden layers tanh, linear outputs), for the two problems. The forward simulations use $m = n$; the classical control at the hardware budget and the inverse problem use $m = 10$. The two coincide at $n = 10$. The learnable ν is not counted.

	HQPINN	classical PINN
features	$\boldsymbol{\xi}$, Eq. (4)	same
$\mathcal{N}_{\text{pre}}$	$3 \to 10 \to n$	$3 \to 10 \to n$
middle block	R_Y, W_{brick}, $\langle Z_q \rangle$	$n \to m \to n$
$\mathcal{N}_{\text{post}}$	$n \to 10 \to 1$	$n \to 10 \to 1$
param., middle	0	$2nm + m + n$
param., total	$21n + 61$	$2n^2 + 23n + 61$ (m=n)
		$42n + 71$ (m=10)
$n = 3, 7, 10$	124, 208, 271	197, 365, 491 (m=10)

TABLE IV. Training hyperparameters of the forward simulations (Figs. 2 and 3), identical for both models.

N_{col} per step	10–100
stencil (h,τ)	$(1/512, 10^{-3})$
optimizer	AdamW, L-BFGS
steps	$\leq 2{\times}10^4 + 10^3$
learning rate	10^{-3}, plateau $\times\frac{1}{2}$
weight decay; gradient clip	10^{-6}; 1
L-BFGS	lr 0.5, history 50
stopping	FDM floor
seeds	0–49

A.3 Simulation protocol

Both models are trained with the same algorithm and hyperparameters (Algorithm 1): AdamW [28] with learning rate 10^{-3} reduced on plateaus, a fresh batch of N_{col} collocation points at every step, up to 2×10^4 steps, followed by at most 10^3 L-BFGS iterations [29] on the last batch. The residual is monitored on a fixed validation set of N_{col} random points, and training stops when it reaches the FDM floor on the same points and with the same stencil (Appendix A.1). The best monitored snapshot

is returned. We sweep $n = 2, \dots, 15$, $N_{\text{col}} \in \{10, 100\}$ and $T \in \{0.1, 1\}$, with 50 seeds per configuration and model (5600 runs), seed s fixing the initialization and the validation set. Complementary sweeps under the same protocol cover N_{col} from 10 to 100, for both models at $n \in \{3, 5, 7, 10, 15\}$. [**TODO:** arquivar no repositório o script que gerou a varredura (`Experiment1Modified`, protocolo `fixed_validation_v2`); `run_qubit_sweep.py --fixed-validation` reproduz o protocolo, conferido em 14 seeds.] The ideal quantum layer is evaluated with the closed form (13), which is exact for the noiseless circuit and differentiable. Neither model has failed runs at $T = 0.1$ (largest MSE 2.2×10^{-3} for the classical PINN and 1.2×10^{-3} for the HQPINN among 1400 runs each).

Algorithm 1. Forward simulation training (classical PINN and ideal HQPINN), long budget.

1: **input:** width n, N_{col}, horizon T, seed s; stencil (h, τ); $K_{\text{A}} = 2 \times 10^4$, $K_{\text{B}} = 10^3$
2: set seed s; initialize Θ; draw a fixed validation set $\mathcal{V}$ of N_{col} points (seed $10^6 + s$)
3: $\mathcal{L}_{\text{FDM}} \leftarrow$ residual of the reference on $\mathcal{V}$ (Appendix A.1)
4: **for** $k = 1, \dots, K_{\text{A}}$: draw N_{col} points; build the $5N_{\text{col}}$ stencil nodes; $\Theta \leftarrow \text{AdamW}(\nabla_\Theta \mathcal{L})$; evaluate $\mathcal{L}(\mathcal{V})$; keep the best snapshot; **stop** if $\mathcal{L}(\mathcal{V}) \le \mathcal{L}_{\text{FDM}}$
5: restore the best snapshot; **for** $k = 1, \dots, K_{\text{B}}$: one L-BFGS iteration on the last batch; evaluate $\mathcal{L}(\mathcal{V})$; **stop** at the target or after 50 iterations without improvement
6: **return** the best snapshot

A.4 The noiseless quantum layer

The first sublayer of W_{brick} acts on the pairs $(1,2), (3,4), \dots$ and the second on $(2,3), (4,5), \dots$ $(n-1$ gates in total).

Remark 1 *A CNOT network acts on the computational basis as a linear map over* GF(2), $\boldsymbol{b} \mapsto M\boldsymbol{b}$. *For the noiseless circuit* (6) *the Heisenberg evolution of* Z_q *is the Pauli string* $\prod_{j \in \mathcal{S}_q} Z_j$ *with* $\mathcal{S}_q = \{j : M_{qj} = 1\} \subseteq \mathcal{C}_q$, *and*

$$y_q(\boldsymbol{\theta}) = \prod_{j \in \mathcal{S}_q} \cos\theta_j, \qquad |\mathcal{S}_q| \le 3. \tag{13}$$

The ideal layer is therefore efficiently classically computable: the noiseless HQPINN is a classical network with cosine-product activations. Its simulation results measure the merit of this architecture, *not a quantum advantage, in line with the general results on classical surrogates of quantum learning models [24]. For angles spread over a full period,* $\text{Var}[y_q] = 2^{-|\mathcal{S}_q|}$, *so deeper entangling layers trade expressivity (longer products) against concentration.*

Proof. CNOT conjugation maps $Z_c \mapsto Z_c$ and $Z_t \mapsto Z_c Z_t$, so Z_q evolves into the string $Z_{\mathcal{S}_q}$ whose support is the qth row of the GF(2) matrix M of the network. On $\bigotimes_j R_Y(\theta_j)|0\rangle$, $\langle Z_j \rangle = \cos\theta_j$ and the state is a product, which gives Eq. (13). For one brickwork layer on $n = 10$ qubits the supports are $\mathcal{S}_1 = \{1\}$, $\mathcal{S}_2 = \{1, 2\}$, $\mathcal{S}_q = \{q-2, q-1, q\}$ for odd $q \ge 3$, $\mathcal{S}_q = \{q-1, q\}$ for even $q \ge 4$, so that, e.g., $y_3 = \cos\theta_1 \cos\theta_2 \cos\theta_3$ and $y_{10} = \cos\theta_9 \cos\theta_{10}$; hence $|\mathcal{S}_q| \le 3$ and $\mathcal{S}_q \subsetneq \mathcal{C}_q$ for most outputs. If the θ_j are independent and uniform on a period, $\mathbb{E}[\cos^2\theta_j] = 1/2$, hence $\text{Var}[y_q] = 2^{-|\mathcal{S}_q|}$ and $\mathbb{E}[(\partial_{\theta_j} y_q)^2] = 2^{-|\mathcal{S}_q|}$.

Appendix B FORWARD PROBLEM: REAL HARDWARE

B.1 Proof of Proposition 1

On hardware, $\hat{y}_q = y_q + \xi_q$ with $\mathbb{E}[\xi_q] = 0$ and $\text{Var}[\xi_q] = (1 - y_q^2)/S \le 1/S$. The hypothesis of Proposition 1 holds on IBM devices because $R_Y(\theta)$ is compiled into fixed microwave pulses and a *virtual* R_Z rotation, so the physical operations, and hence their errors, do not depend on θ. Equation (11) has three consequences. (i) The quantum layer injects a finite Fourier structure into the latent representation [30]: the pre-network learns where to place the angles, and the circuit multiplies harmonics of different latent coordinates, $\cos\theta_a \cos\theta_b = \frac{1}{2}[\cos(\theta_a + \theta_b) + \cos(\theta_a - \theta_b)]$, in the spirit of Fourier-feature networks [31, 32]. (ii) Noise does not change the function class, only the coefficients; this is what makes a calibrated surrogate of the device possible. (iii) The quantum layer has no trainable parameter, so what remains of the barren-plateau question is the possible concentration of the map $\boldsymbol{\theta} \mapsto \boldsymbol{y}$ itself, which is controlled by the size of the light cones (Remark 1). The fixed layer connects the HQPINN to fixed-substrate approaches such as quantum feature maps [18], quantum reservoir computing and extreme learning machines [19, 20] and quanvolutional networks [21], none of which has been used to minimize a PDE residual.

Trigonometric structure. For a single rotation,

$$R_Y(\theta)\,\rho\,R_Y^\dagger(\theta) = \tfrac{1}{2}(\rho + Y\rho Y) + \tfrac{1}{2}\cos\theta\,(\rho - Y\rho Y) - \tfrac{i}{2}\sin\theta\,[Y, \rho], \tag{14}$$

so the map $\rho \mapsto R_Y(\theta)\rho R_Y^\dagger(\theta)$ is an affine combination of $\{1, \cos\theta, \sin\theta\}$ with θ-independent superoperators. All other operations—CNOTs, noise channels, readout confusion matrices and the trace with Z_q—are linear and θ-independent. Since each angle appears in exactly one gate, y_q is a sum of products in which every θ_j contributes exactly one factor in $\{1, \cos\theta_j, \sin\theta_j\}$, which is Eq. (11).

Light cone. In the Heisenberg picture $y_q = \text{Tr}[\rho_0\,\mathcal{E}_{\boldsymbol{\theta}}^\dagger(Z_q)]$. The adjoint of a trace-preserving channel is unital, so every gate (and every gate-local noise channel) whose support does not intersect the current support of the evolved operator leaves it unchanged. The support grows only through gates that touch it; the set of qubits

reached is $\mathcal{C}_q$, and rotations outside $\mathcal{C}_q$ drop out. Going backwards through one brickwork layer, the second sublayer adds at most the partner of q and the first sublayer at most one qubit on each side, so $|\mathcal{C}_q| \leq 4$; for D layers, $|\mathcal{C}_q| \leq \min(n, 4D)$. For $n = 10$ the cones are $\{1,2\}$, $\{1,\dots,4\}$ (twice), $\{3,\dots,6\}$ (twice), $\{5,\dots,8\}$ (twice), $\{7,\dots,10\}$ (twice) and $\{9,10\}$. Crosstalk or non-local noise invalidates this bound and enlarges the effective cone.

Conditioning of the calibration. With $\theta_j \sim \mathcal{U}(-\pi,\pi)$ i.i.d., the features $\prod_{j\in\mathcal{C}_q} g_{\alpha_j}(\theta_j)$ are orthogonal, with $\mathbb{E}[g_0^2] = 1$ and $\mathbb{E}[g_c^2] = \mathbb{E}[g_s^2] = 1/2$. The Gram matrix of the least-squares problem is therefore diagonal in expectation, with condition number $2^{|\mathcal{C}_q|} \leq 16$, and the standard error of a coefficient with k non-constant factors is $\simeq \sigma\, 2^{k/2}/\sqrt{N_{\rm cal}}$. With $N_{\rm cal} = 1024$ samples and $\sigma \simeq 1/32$, the highest-order coefficients ($k = 4$) are determined to $\simeq 4 \times 10^{-3}$, well below the shot noise of a single measurement.

B.2 The loop on a device

a Why a surrogate. The device returns $\hat{\boldsymbol{y}}$ but not $\partial \boldsymbol{y}/\partial\boldsymbol{\theta}$. Parameter-shift rules [16, 17, 33] give unbiased derivatives at the price of $2n$ extra circuits per evaluation point, and SPSA [34, 35] gives a single random directional derivative per step. If the five stencil nodes were measured independently, the shot noise σ_u of u would also enter $\partial_x^2 u$ amplified by $\sqrt{6}/h^2 \approx 6.4 \times 10^5$ (Appendix B.4).

b The three stages. Calibration: before training, one job evaluates the circuit at random angles $\theta_j \sim \mathcal{U}(-\pi,\pi)$ on every n-qubit chain of the chip in parallel; the coefficients of Eq. (11) are obtained by linear least squares, which yields a surrogate $\hat{f}(\boldsymbol{\theta})$ of the device with its average noise built in [36]. *Training:* Eq. (12) at each epoch. At the centre $\tilde{\boldsymbol{y}} = \hat{\boldsymbol{y}}^{\,\rm QPU}$, so the value of u entering the nonlinear term $u\,\partial_x u$ is the measured one; across the stencil the variation of $\tilde{\boldsymbol{y}}$ is that of the smooth surrogate; and the gradient is $\partial\tilde{\boldsymbol{y}}/\partial\boldsymbol{\theta} = \partial\hat{f}/\partial\boldsymbol{\theta}$, available analytically. Every $K_{\rm fit}$ epochs the surrogate is refitted with the calibration data and all measurements accumulated so far, so that it tracks the device in the region of angles actually visited by the network. *Validation:* a final job evaluates $u(x,T)$ with the trained network and the circuit run on the device. Because the surrogate has the exact functional form of the device response, the method does not approximate the circuit; its only bias is the statistical error of the fitted coefficients and the part of the device behaviour that the time-independent, chip-averaged model (11) cannot capture (chain-to-chain variation, drift, crosstalk). The construction applies to any circuit whose light cones are small enough for the $3^{|\mathcal{C}_q|}$ coefficients of Eq. (11) to be calibrated, which bounds the entangling depth to a few brickwork layers, and it is compatible with error mitigation schemes that are affine in the measured data (e.g., readout mitigation [37], Pauli twirling, linear zero-noise extrapolation [38]).

Algorithm 2. Forward HQPINN training with the quantum processor in the loop.

input: chains $\{\mathcal{Q}_b\}_{b=1}^{B}$, shots S, epochs K, $N_{\rm col}$, refit period $K_{\rm fit}$, schedule η_k
calibration job: $\lceil 1024/B \rceil$ random angle sets on all chains $\to (\boldsymbol{\theta}, \hat{\boldsymbol{y}})$; screen chains; fit $\hat{f}$ [Eq. (11)]
set seed; initialize $\Theta = (\boldsymbol{\phi}, \boldsymbol{\psi})$
for $k = 1, \dots, K$:
  draw $N_{\rm col}$ centres; stencil nodes $\mathcal{S}$; $\boldsymbol{\theta} \leftarrow \mathcal{N}_{\rm pre}(\boldsymbol{\xi}(\mathcal{S}))$
  job k: measure $\hat{\boldsymbol{y}}_c$ at the centres; centre p runs on chain $(p + k) \bmod B$
  $\tilde{\boldsymbol{y}} \leftarrow \hat{f}(\boldsymbol{\theta}) + \mathrm{sg}[\hat{\boldsymbol{y}}_c - \hat{f}(\boldsymbol{\theta}_c)]$ on all nodes [Eq. (12)]
  $u \leftarrow$ Eq. (4); $\mathcal{L} \leftarrow$ Eq. (8); backward through $\hat{f}$; clip $\|\nabla\| \leq 1$; AdamW step with η_k
  if $k \bmod K_{\rm fit} = 0$: refit $\hat{f}$ with the calibration and all measurements so far
validation job: $u(x,T)$ on 200 points, each measured on every chain and averaged

The learning rate is set at every epoch, $\eta_k = \max\{0.1 - 0.01\lfloor (k-1)/10 \rfloor,\, 0.01\}$, and the AdamW state is kept across epochs.

TABLE V. Hyperparameters at the hardware budget: `ibm_berlin`, FakeBerlin, classical control and hardware-budget simulations (Fig. 4, Appendix B.5).

$N_{\rm col}$ per epoch	10
stencil (h, τ)	$(1/512, 10^{-3})$
optimizer; steps	AdamW; 110
learning rate	0.1 to 0.01
weight decay; gradient clip	10^{-6}; 1
stopping	fixed budget
shots S	1024
calibration	1024 samples
refit $K_{\rm fit}$	10 epochs
seeds	0 (QPU), 0–4

B.3 The run at $T = 0.1$

The hardware experiment ran on `ibm_berlin`, an IBM Nighthawk processor with 120 transmon qubits on a square lattice and native CZ gates, on 8 October 2026, within a single dedicated runtime `Session`. The circuit (6) with $n = 10$ was laid out on 8 disjoint chains of 10 coupled qubits (Fig. 7), selected from the current calibration data (12 candidate chains, 4 discarded for a high error score), and the 8 copies were transpiled into a single packed circuit of depth 12 with 72 CZ gates and no SWAPs. All expectation values were estimated with $S = 1024$ shots (precision 1/32) and *without* error suppression or mitigation (raw counts). The run consisted of one calibration job (128 parameter sets on 8 chains, 1024 samples per output), 110 training jobs (one per epoch, the 10 stencil centres distributed over the chains) and one

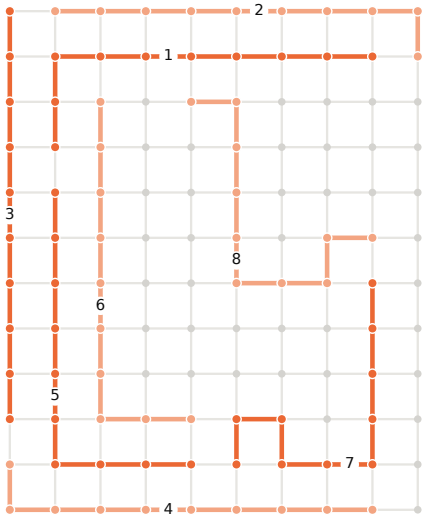


FIG. 7. The 120-qubit square lattice of `ibm_berlin` and the 8 disjoint 10-qubit chains on which the circuit ran in parallel (alternating shades and numbers distinguish neighbouring chains).

validation job (200 points, each measured once on every chain and averaged), i.e., 112 jobs, 548 parameter sets and 5.6×10^5 shots in 54 min of wall-clock time. Training used AdamW with learning rate 0.1 decreased by 0.01 every 10 epochs down to 0.01, gradient-norm clipping at 1 and $K_{\text{fit}} = 10$. The hardware seed (seed 0) is the best of an earlier FakeBerlin batch of five seeds. Device characteristics are listed in Table VI.

a Chains. All disjoint chains of n physically coupled qubits are searched on the coupling map with a randomized greedy depth-first search that starts from the qubits with fewest free neighbours (100 trials; the largest set is kept). Each chain is scored by the sum of its readout and $\sqrt{X}$ errors and of the CZ errors of its links, and chains with a score above $0.05\,n$ are discarded; on `ibm_berlin` 8 of 12 candidate chains were kept (rejected scores 0.51, 0.57, 1.27, 3.83). The circuit is laid out on all kept chains in a single packed circuit, so that one parameter set yields one sample per chain; transpilation with an initial layout on the chains produced no SWAP (72 CZ, depth 12).

TABLE VI. `ibm_berlin` on 8 October 2026: the 80 qubits and 72 couplers used (median and range).

quantity	median	range
T_1	322 μs	110–528 μs
T_2	255 μs	34–510 μs
readout error	1.7%	0.5–6.3%
$\sqrt{X}$ error	1.7×10^{-4}	$(0.7\text{–}9.9) \times 10^{-4}$
CZ error	1.7×10^{-3}	$(0.6\text{–}14) \times 10^{-3}$
qubits used / total	80 / 120 (8 chains × 10)	
packed circuit	depth 12, 72 CZ, no SWAP	
jobs / shots / time	112 / 5.6×10^5 / 54 min	

b Calibration and screening. The calibration job evaluates 128 random angle sets on the 8 chains (1024 samples per output). Since the chains are physically different, each chain is also screened: its RMS misfit to the chip-averaged surrogate is computed, and chains above three times the median are discarded. On `ibm_berlin` the misfits were 0.032–0.080 and all chains were kept. The surrogate is a single set of coefficients for the whole chip; the per-output fit RMS was 0.025–0.095 (mean 0.039, shot noise 0.031) and the RMS difference between the device and the ideal formula 0.064 (FakeBerlin: 0.028 and 0.039).

c Training jobs. The 10 stencil centres of each epoch fill 2 parameter sets on the 8 chains; centre p runs on chain $(p+k) \bmod 8$ at epoch k, which rotates the points over the chains and prevents a systematic assignment of points to poor chains. The surrogate is refitted every 10 epochs with the calibration data and all training measurements (1124 to 2124 samples). An epoch took 16 s on average, dominated by job latency inside the session; on the emulator, 3.5 s.

d Validation. The validation job evaluates the 200 points at $t = T$ with 8 replicas, one on every chain, and averages $\hat{\boldsymbol{y}}$ over the replicas before the post-network, so the average matches the chip-averaged surrogate. Three evaluations of the same weights give $\epsilon_{L_2} = 5.7\%$ with the circuit on the device, 5.5% with the surrogate and 6.0% with the ideal formula.

e Offline replay. The run log contains the initial weights, the calibration data and, for every epoch, the collocation points, the angles sent to the device and the measured $\hat{\boldsymbol{y}}$. Replaying Algorithm 2 with the recorded measurements in place of the device reproduces the trained weights, the training losses and the angles of every epoch exactly (maximum difference zero, for both `ibm_berlin` and FakeBerlin), which we use to evaluate the network after each epoch [Fig. 4(b)] without additional quantum resources. This holds on the software stack of the run (Windows, PyTorch 2.14.1). On Linux, with the same PyTorch version and the same recorded measurements, the angles differ by 6×10^{-8} at the first epoch: the rounding error, amplified by the finite differences of a single-precision residual, grows to order one, and the replay ends at another network, with a surrogate MSE of 6.3×10^{-3} ($\epsilon_{L_2} \simeq 12\%$) instead of 1.4×10^{-3}. The training is therefore chaotic at this learning rate, and the difference lies within the seed-to-seed spread of the noiseless model. It is one more reason to read a single run with care. The same happens to the noiseless simulations of Appendix B.5, which on Linux give a median of 8.3% (4.6–13.4%), against the recorded 9.1% (3.9–15.1%). [**TODO:** avaliar se esta sensibilidade deve ir para o texto principal, e se vale repetir o treino em dupla precisão.]

f The run at $T = 1$. [**TODO:** A rodada ainda não existe; caso proposto em `QPU_CASES`: $n = 10$, $N_{\text{col}} = 20$,

seed 0, 110 épocas. Acrescentar aqui os detalhes técnicos dela.]

B.4 Straight-through estimator and finite differences

Stencil formulas and truncation. On the cross stencil,

$$\partial_t u \approx \frac{u(x,t+\tau)-u(x,t-\tau)}{2\tau}, \qquad \partial_x u \approx \frac{u_+ - u_-}{2h},$$
$$\partial_x^2 u \approx \frac{u_+ - 2u_0 + u_-}{h^2}, \qquad u_\pm = u(x\pm h, t), \tag{15}$$

with truncation errors $\frac{\tau^2}{6}\partial_t^3 u$, $\frac{h^2}{6}\partial_x^3 u$ and $\frac{h^2}{12}\partial_x^4 u$; for $h = 1/512$ and $\tau = 10^{-3}$ these are negligible against the other errors of the method.

Noise amplification. If each node carried independent noise of standard deviation σ_u (shot noise $\leq 1/\sqrt{S}$ in y_q propagated through $\mathcal{N}_{\text{post}}$), the derivative estimates would have

$$\text{std}[\partial_x u] = \frac{\sigma_u}{\sqrt{2}\,h}, \qquad \text{std}[\partial_x^2 u] = \frac{\sqrt{6}\,\sigma_u}{h^2}. \tag{16}$$

With $h = 1/512$ and $\sigma_u \sim 10^{-2}$, $\text{std}[\partial_x^2 u] \sim 6 \times 10^3$, three orders of magnitude above $|\partial_x^2 u|$ away from the front. Enlarging h to make this tolerable would introduce a truncation bias in the residual.

Shared offset. Eq. (12) assigns the same correction $\Delta_c = \hat{\boldsymbol{y}}^{\text{QPU}}(\boldsymbol{\theta}_c) - \hat{f}(\boldsymbol{\theta}_c)$ to the five nodes of the stencil. Writing $\tilde{\boldsymbol{y}}_\pm = \hat{f}(\boldsymbol{\theta}_\pm) + \Delta_c$ and expanding $\mathcal{N}_{\text{post}}$,

$$u_+ - u_- = \tfrac{t}{T} J(\hat{f}_c + \Delta_c)\left[\hat{f}(\boldsymbol{\theta}_+) - \hat{f}(\boldsymbol{\theta}_-)\right] + \mathcal{O}(h^3), \tag{17}$$

where J is the Jacobian of $\mathcal{N}_{\text{post}}$. The noise enters only through the evaluation point of J, so it perturbs the derivatives by a relative amount $\mathcal{O}(\|J'\|\,\|\Delta_c\|)$ instead of $\mathcal{O}(\sigma_u/h^2)$. The measured value enters the factor u of $u\,\partial_x u$ directly.

Gradient bias. The backward pass uses $\partial\hat{f}/\partial\boldsymbol{\theta}$ instead of the (inaccessible) derivative of the device response. By Proposition 1 both are derivatives of trigonometric polynomials of the same form, so the bias is the derivative of the coefficient error and of the unmodelled part of the response. Parameter-shift rules would give an unbiased but noisy estimate of the same quantity from $2n$ additional circuits per point [17, 33]; the surrogate replaces them by one regression over all measurements, in the spirit of model-based optimizers for variational algorithms [36, 39, 40].

B.5 Controls at the hardware budget

All controls use the protocol of the run at $T = 0.1$ (110 epochs, $N_{\text{col}} = 10$, same learning-rate schedule and collocation stream per seed): they follow the loop of Algorithm 2 with the device replaced by the classical middle block, by the ideal layer (with or without sampled shot noise), or by the emulator.

a Classical control. The classical PINN is trained with exactly the hardware protocol for $n = 1, \ldots, 10$ and seeds 0–4 (50 runs, $m = 10$ in Table III). It ends between 4.6% and 35.9% (median 24.7%), and 28% of its runs end above the error of a network that does not evolve the initial condition. For $n = 10$ the best of ten schedules (the step decay of the HQPINN rescaled to η_0, or a cosine decay from η_0 to $\eta_0/10$, with $\eta_0 \in \{1, 3, 10, 30, 100\} \times 10^{-3}$), selected on the test error itself, is a cosine decay from $\eta_0 = 0.03$; it lowers the median ϵ_{L_2} from 31.1% to 11.4% (6.2–19.5%, MSE 5.8×10^{-3}). For $n = 3$ the schedule of the HQPINN is already the best (13.7%). Replacing tanh by sin in the classical middle block at $n = 10$ lowers the median ϵ_{L_2} from 31.1% to 20.9% (7.0–27.2%) under the schedule of the HQPINN and from 11.4% to 10.6% (8.7–12.5%) under the best classical schedule: a periodic activation closes part of the gap at a common schedule and has little effect once the schedule is tuned. [**TODO:** repetir o controle com `seno=True` no protocolo longo ($T = 1$), onde está a vantagem.]

b Hardware-budget simulations. To separate the effect of noise from that of the short training, the HQPINN is trained offline, seeds 0–4, with (i) the ideal layer on all stencil nodes, reaching ϵ_{L_2} between 3.9% and 15.1% (median 9.1%; 5.5% for seed 0), and (ii) the ideal layer plus binomial shot noise with $S = 1024$ at the stencil centres, propagated with Eq. (12) and a perfect surrogate, which does not degrade it (3.8%–16.2%, median 5.5%).

c Noisy emulation. The device is emulated with the `FakeBerlin` noise model of Qiskit [41], which reproduces the calibrated gate errors, T_1/T_2 relaxation, readout errors and coupling map of `ibm_berlin` but not crosstalk or temporal drift. Each 10-qubit chain is simulated exactly as a density matrix and $S = 1024$ shots are sampled. The emulated training uses the same physical chains, seed, collocation stream and protocol as the real run (Algorithm 2), and gives 4.1% for seed 0 and 6.9%–8.1% for seeds 1–4. [**TODO:** Seeds 1–4: valores da versão anterior do texto; os arquivos brutos não estão no repositório nem no histórico. Seed 0 e os 4,8% abaixo foram conferidos em `data/quantum_ibm_faker/T_0.1/`.] The network trained on FakeBerlin and evaluated on `ibm_berlin` (one prediction job) gives 4.8%: the noise model transfers to the real device with a small loss. With one seed each, training on the device (5.7%) and on the emulator (4.8%

TABLE VII. Circuit evaluations per epoch for the gradient of the residual loss ($N_{\text{col}} = 10$, five stencil nodes per point, $n = 10$ angles per node). Parameter shift, SPSA and finite differences require measuring every stencil node, so the shot noise of $\partial_x^2 u$ is amplified by $\sim h^{-2}$.

method	circuits/epoch	noise in $\partial_x^2 u$
surrogate + STE (this work)	10 (1 job)	not amplified
parameter shift	$\simeq 1050$	$\times\, h^{-2}$
SPSA	$\simeq 100$	$\times\, h^{-2}$
finite differences in $\boldsymbol{\theta}$	$\simeq 1050$	$\times\, h^{-2}$

on the device) both lie within the seed-to-seed spread of the noiseless model, so neither strategy can be declared superior.

Appendix C INVERSE PROBLEM: SIMULATION

C.1 Exact solution

The Cole–Hopf transformation [42, 43] $u = -2\nu\,\partial_x \ln\varphi$ maps Eq. (1) onto the heat equation $\partial_t\varphi = \nu\,\partial_x^2\varphi$. For $u(x,0) = \sin(2\pi x)$,

$$\varphi(x,0) = \exp\!\left[-\frac{1}{2\nu}\int_0^x \sin(2\pi s)\,ds\right] \propto e^{z\cos(2\pi x)}, \quad (18)$$

with $z = 1/(4\pi\nu)$. Using $e^{z\cos\vartheta} = I_0(z) + 2\sum_{m\geq 1} I_m(z)\cos m\vartheta$, with I_m the modified Bessel functions, each Fourier mode decays as $e^{-4\pi^2 m^2\nu t}$ and

$$u(x,t) = \frac{8\pi\nu\sum_{m\geq 1} m\,I_m(z)\,e^{-4\pi^2m^2\nu t}\sin(2\pi m x)}{I_0(z) + 2\sum_{m\geq 1} I_m(z)\,e^{-4\pi^2m^2\nu t}\cos(2\pi m x)}. \quad (19)$$

The series is evaluated in double precision with exponentially scaled Bessel functions $I_m(z)e^{-z}$ (the common factor cancels) and $m < 4z+64$ terms. It reproduces the initial condition to machine precision and satisfies Eq. (1) to the accuracy of a finite-difference check ($\sim 10^{-5}$); for $\nu = 0.01$, $\max_x u \simeq 0.961$ at $T = 0.1$ and 0.331 at $T = 1$. The series loses precision for $z \gtrsim 16$ ($\nu \lesssim 5\times 10^{-3}$), where a Gauss–Hermite integral representation [25] must be used instead.

C.2 Training algorithm

Both models are trained with Algorithm 3; for the classical model the branches marked HQPINN are skipped.

Algorithm 3. Inverse training of the classical PINN and the HQPINN.

input: epochs K, N_{col}, stencil (h,τ), schedules η_k, β_k, ρ_k, weight w_d; HQPINN: shots S, refit period K_{fit}
HQPINN: calibration job, shared by all runs of a size $\to$ fit $\hat{f}$ [Eq. (11)]
set seed; draw $\nu_0 \sim \mathcal{U}(0,1)$; initialize Θ
for $k = 0, \ldots, K-1$:
  draw N_{col} centres; stencil nodes; data $u_i^\star$ at the centres
  HQPINN: if $k > 0$ and $k \bmod K_{\mathrm{fit}} = 0$: refit $\hat{f}$; *one job*: measure $\hat{\boldsymbol{y}}_c$ at the centres; keep the offsets of Eq. (12)
  repeat 16 times: forward pass; $A_i, B_i \leftarrow$ finite differences; $\Theta \leftarrow \mathrm{AdamW}(\nabla_\Theta \mathcal{L}_k; \eta_k)$ with ν_k fixed, Eq. (8)
  $\nu^\star \leftarrow \mathrm{wmed}(\{A_i/B_i\}; \{|B_i|\})$ from the last forward pass, clipped to $[10^{-4}, 10]$
  $\log\nu_{k+1} \leftarrow \log\nu_k + \beta_k(\log\nu^\star - \log\nu_k)$
return $\hat{\nu} = \nu_K$, u_Θ; validation job on $u(x,T)$

a Collocation and residual. At every epoch a fresh batch of N_{col} points is drawn uniformly, $x_i \in [0,1)$ and $t_i \in [\tau, T-\tau]$. The *same* points are used for the residual and for the data term, so that each evaluation of the model enters both. The stencil is that of Appendix B.4, with nodes wrapped modulo 1 in x. The noise amplification described there would be worse here, because it would enter B and hence $\nu^\star$ directly. At the centre $\tilde{\boldsymbol{y}} = \hat{\boldsymbol{y}}^{\mathrm{QPU}}$, so the data term and the factor u of $u\,\partial_x u$ use the measured value.

b Loss and network step. The network parameters $\Theta = (\boldsymbol{\phi}, \boldsymbol{\psi})$ [plus $\boldsymbol{\chi}$ in the classical model] are updated with AdamW [28] on Eq. (8) with $p = 1$ and ν_k held fixed. The residual weight $\rho_k = \min\{1, (k+1)/K_{\mathrm{ramp}}\}$ is ramped up so that the networks first fit the data, and the data weight $w_d \gg 1$ is fixed. There are 16 steps per epoch on the batch; in the HQPINN all of them reuse the single measurement of the epoch. The learning rate follows a cosine decay [44] over the K epochs and the gradient norm is clipped. The final state is that of the last epoch, with no L-BFGS and no snapshot.

C.3 Variable projection for the viscosity

The estimate (9) comes from the same forward pass as the network step and is clipped to $[10^{-4}, 10]$. The relaxation (10) reads $\nu_{k+1} = \nu_k^{1-\beta_k}(\nu_k^\star)^{\beta_k}$, with β_k decaying from β_0 to β_K with a cosine schedule. It is an exponentially weighted geometric average of the batch estimates; it keeps $\nu > 0$, is scale invariant, and its effective memory $\sim 1/\beta_k$ grows during training, damping the noise of late batches. The initial value ν_0 is drawn uniformly in $(0,1)$.

Weighted-median solution. Write $\sum_i |A_i - \nu B_i| = \sum_i w_i |r_i - \nu|$ with $r_i = A_i/B_i$ and $w_i = |B_i|$ (points with $B_i = 0$ do not depend on ν and are discarded). This is a convex piecewise-linear function of ν whose subgradient is $\sum_{r_i<\nu} w_i - \sum_{r_i>\nu} w_i + [-1,1]\sum_{r_i=\nu} w_i$. It contains zero when the weight on each side of ν is at most half of the total, i.e., at the weighted median of $\{r_i\}$. Points with small curvature $|B_i|$, for which the ratio A_i/B_i is ill-conditioned, receive proportionally small weight. Under the L_2 norm the minimizer would instead be $\nu^\star = \sum_i A_i B_i / \sum_i B_i^2$, which is not robust to outliers. In the implementation the ratios are sorted, the cumulative weight is formed, and the first ratio at which it reaches half of the total is returned.

C.4 Adjusted training at $T = 1$

At $T = 1$ the algorithm above fails to represent the solution: with the hard constraint of Eq. (4) the network has to cancel $\sin(2\pi x)$ linearly in t while forming the front, and after 110 epochs the classical model reconstructs $u(x,T)$ with a relative L_2 error of 0.2–0.4. For this case only, and identically for both models and both budgets, three changes are made: (i) the network output

is u itself, and the initial condition enters as $\mathcal{L}_{\text{ic}}$ in Eq. (8), the mean absolute error at N_{col} random points of $t = 0$ per epoch with weight $w_0 = 10$ (for the HQPINN these points are measured in the same job as the collocation centres, so the number of jobs is unchanged); (ii) during the first $K_0 = 10$ epochs only the data term is used and ν is not updated, after which the residual weight is ramped up; (iii) the learning rate of the networks is kept constant and divided by 1.5 whenever the epoch loss has not decreased by 1% for 5 epochs, with a floor of 1% of its initial value, instead of following a cosine decay. These changes were selected on the classical model among 147 variants of the training (other optimizers, including L-BFGS; loss-balancing schemes [4]; learning-rate rules driven by the loss; warm-up lengths), on 60 runs with seeds not used in this paper: the median ϵ_ν dropped from 38.5% to 12.7% and the median L_2 error of $u(x, T)$ from 0.23 to 0.067. On a proxy of the device (calibrated surrogate plus shot noise) the same changes reduce the median ϵ_ν of the HQPINN from 32.7% to 20.7%; no other change of its hyperparameters gave a significant further gain, whereas more collocation points per epoch did (10.5% with $N_{\text{col}} = 20$, 4.2% with $N_{\text{col}} = 40$). Here $N_{\text{col}} = 10$ is kept for both models. [**TODO:** Estes números dos testes de treino vêm de `data/Inverso_110/README.md`: os dados e scripts dos testes foram apagados e não estão no histórico, então não foram reconferidos.]

C.5 Hyperparameters

The hyperparameters were selected independently for each model with one protocol, for $\nu_{\text{true}} = 0.1$ and $T = 0.1$: search on seeds 100–105, selection on seeds 200–219, and test on seeds 0–2. The same values are used at both budgets, and the cosine decays stretch over the K epochs. With the hyperparameters of the HQPINN, the classical model reaches there a median ϵ_ν of 6.7% on the selection seeds, against 4.0% with its own.

C.6 Emulators

The devices are the noise models of `ibm_berlin` (`FakeBerlin`) and `ibm_aachen` (`FakeAachen`), simulated with Qiskit Aer [41]; no job was sent to IBM (Table IX). They reproduce the calibrated gate and readout errors and the coupling map, but not crosstalk, temporal drift or the error mitigation of the runtime service. The simulator does not fix its seed, so rerunning a case gives slightly different numbers. A calibration is cached and shared by all runs of a size, and it is the record of the chains used: for `FakeBerlin` the chains come from the chain search available when the runs were made, and for `FakeAachen` they were computed from the error table of the backend (all paths of n coupled qubits, scored as in Appendix B.3, then disjoint chains from the best to the worst). The misfit of the calibration is at or below the shot-noise bound $1/\sqrt{S} = 3.125\times10^{-2}$. A run costs $K+1$ jobs besides the shared calibration: one per epoch and one validation job, which evaluates $u(x, T)$ on 100 points at $t = T$ with one replica of each point per chain, and the replicas are averaged. Three versions are reported: the model with the circuit run on the emulator, the model with the surrogate it was trained through, and the same weights with the ideal circuit.

TABLE VIII. Hyperparameters of the inverse problem. The last three rows apply to $T = 1$ only (Appendix C.4).

	HQPINN	classical
epochs K	110, 500	110, 500
N_{col}	10	10
stencil (h, τ)	$(1/512,\ 10^{-3})$	same
lr, initial	3.07×10^{-2}	2.53×10^{-2}
lr, final (cosine, $T = 0.1$)	1.2×10^{-3}	3.2×10^{-4}
AdamW steps per epoch	16	16
AdamW (β_1, β_2)	(0.77, 0.99)	(0.73, 0.99)
weight decay	6.4×10^{-6}	9.3×10^{-4}
gradient-norm clip	10	10
data weight w_d	85.8	37.8
residual ramp K_{ramp}	20	10
β_k, initial (final)	0.97 (0.128)	0.50 (0.163)
$\nu^\star$ clip	$[10^{-4}, 10]$	same
shots S	1024	–
surrogate refit K_{fit}	10	–
initial-condition weight w_0	10	10
warm-up K_0 (data only, ν frozen)	10	10
lr on plateau (patience; factor)	5; 1/1.5	5; 1/1.5

TABLE IX. Emulators and cost. Backend data over the whole chip, from the snapshot in the `FakeBerlin` and `FakeAachen` objects of `qiskit-ibm-runtime` 0.51 (medians). Time per run, median, with 12 runs sharing 16 CPU cores; the classical model takes about 3 s at $K = 110$ and 20–24 s at $K = 500$.

	`FakeBerlin`	`FakeAachen`
processor; qubits	Nighthawk; 120	Heron; 156
readout error	2.1×10^{-2}	7.4×10^{-3}
$\sqrt{X}$ error	2.1×10^{-4}	1.9×10^{-4}
CZ error	2.2×10^{-3}	1.5×10^{-3}
T_1; T_2 (μs)	324; 248	232; 255
chains, $n = 3, 7, 10$	33, 14, 9	41, 14, 10
calibration misfit (RMS)	0.032–0.034	0.027–0.028
epochs K	110	500
jobs per run	111	501
time, $n = 3, 7, 10$ (s)	374, 436, 1182	1001, 1318, 3608

C.7 Results at 110 epochs and per size

At $K = 110$ on `FakeBerlin`, at $T = 0.1$ both models approach ν_{true} within ~ 20 epochs and then keep improving as β_k decays: the median error of the HQPINN goes

from 21.5% at epoch 60 and 7.9% at epoch 80 to 6.0% at the end, and that of the classical model from 26.1% and 12.2% to 7.3%. The HQPINN has the smaller tail (90th percentile 15% against 20%) and is more uniform across sizes, and both models slightly overestimate ν (in 79% of the runs for the HQPINN and 76% for the classical model). At $T = 1$ the HQPINN at $n = 7$ has a median ϵ_ν of 15.9%, below the 17.1–28.5% of the classical model, whereas $n = 3$ and 10 give 36.9% and 39.6%. The best runs of either model reach $\epsilon_\nu \simeq 1$–2% in both cases at this budget, and the classical model at $T = 1$ has a median ϵ_ν of 10.1% on the first three seeds alone, half of its value over the 33 runs, so comparisons require many seeds. Table X gives the results per size at both budgets.

TABLE X. Relative error of the learned viscosity, ϵ_ν (median / mean / max over seeds 0–10, in %), and median relative L_2 error of $u(x, T)$, per size n; $\nu_{\text{true}} = 0.01$.

	$T = 0.1$		$T = 1$	
	ϵ_ν (%)	ϵ_{L_2}	ϵ_ν (%)	ϵ_{L_2}
$K = 500$ (FakeAachen)				
HQPINN, $n = 3$	3.2 / 4.0 / 13.4	0.021	41.2 / 41.7 / 67.1	0.089
HQPINN, $n = 7$	3.0 / 3.0 / 7.7	0.013	27.8 / 28.1 / 63.6	0.070
HQPINN, $n = 10$	2.6 / 2.6 / 7.4	0.009	12.8 / 22.4 / 75.0	0.065
classical, $n = 3$	2.6 / 3.3 / 10.2	0.006	16.9 / 22.6 / 62.5	0.044
classical, $n = 7$	2.3 / 2.9 / 6.7	0.005	33.7 / 31.4 / 51.7	0.046
classical, $n = 10$	1.4 / 3.1 / 12.5	0.006	14.6 / 24.0 / 62.8	0.041
$K = 110$ (FakeBerlin)				
HQPINN, $n = 3$	6.5 / 9.6 / 25.1	0.034	36.9 / 37.1 / 67.5	0.115
HQPINN, $n = 7$	3.8 / 4.4 / 9.1	0.021	15.9 / 18.5 / 38.6	0.127
HQPINN, $n = 10$	5.3 / 6.1 / 15.7	0.017	39.6 / 41.2 / 71.1	0.100
classical, $n = 3$	13.2 / 12.8 / 29.8	0.015	17.1 / 19.4 / 43.1	0.074
classical, $n = 7$	4.3 / 6.3 / 22.7	0.014	28.5 / 27.9 / 68.5	0.076
classical, $n = 10$	11.5 / 10.2 / 17.8	0.015	21.5 / 21.9 / 44.7	0.070

C.8 Older long-budget sweeps

Earlier sweeps with a long budget (20 000 AdamW steps plus L-BFGS, $\nu_{\text{true}} = 0.01$, with 100 data points at $t = T$ only) give context but are not comparable with the results above, nor with each other. In the classical PINN (50 seeds, no early stopping), the median ϵ_ν at $n = 3, 7, 10$ is 1.6–1.9% for $T = 0.1$, $N_{\text{col}} = 100$, 11–14% for $T = 0.1$, $N_{\text{col}} = 10$, and above 100% at $T = 1$ (111–122% with $N_{\text{col}} = 100$, 274–302% with $N_{\text{col}} = 10$). The noiseless hybrid runs of that period used another circuit (a linear CNOT network), a stopping target and 10 seeds, with some configurations incomplete: their median is 6.6–7.6% for $T = 0.1$, $N_{\text{col}} = 100$ and at best 62% at $T = 1$. Under the rule of the rest of this paper, the two are not compared. Figure 8 shows the mean squared error of ν versus N_{col} for widths up to $n = 10$. At $T = 0.1$ that of the classical PINN falls from 1.7×10^{-4} ($n = 3$) and 3.1×10^{-6} ($n = 7$) at $N_{\text{col}} = 10$ to 4.6×10^{-8} and 7.0×10^{-8} at $N_{\text{col}} = 100$, while that of the older HQPINN stays near 5×10^{-7} from $N_{\text{col}} = 20$ on. At $T = 1$ the older HQPINN has the lower error, 4.4×10^{-5} against 3.4×10^{-4} at $n = 7$, $N_{\text{col}} = 100$, but both are far from identifying ν.

The contrast with Table II at $T = 1$ suggests, but does not establish, the value of data at the collocation points themselves and of the adjusted training.

Appendix D INVERSE PROBLEM: REAL HARDWARE

[**TODO:** Nenhuma rodada real do problema inverso foi executada: este apêndice descreve o que está pronto e o que falta decidir. Substituir pelos detalhes da rodada quando ela existir, como no Apêndice B.3.]

D.1 Protocol

The script of the emulated runs also drives a real processor. It uses the machinery of Appendix B.2 (chain search and screening, packed circuit in a runtime `Session`, chip-averaged surrogate, refit every 10 epochs, final validation on the device). The differences with respect to the forward run are the following. (i) The measured values at the centres also enter the data term. (ii) Each measurement serves 16 AdamW steps, all with the same offsets, so the number of steps does not change the number of jobs. (iii) At $T = 1$ the N_{col} points of the initial-condition term are measured in the same job as the centres (20 points per job). (iv) The validation job uses 100 points instead of 200.

D.2 Proposed cases and cost

The cases listed in the code (`INVERSE_QPU_CASES`) are a proposal: $n = 10$, $N_{\text{col}} = 10$, 110 epochs, seed 0, at $T = 0.1$ and $T = 1$, the same size and budget as the forward run. Table XI gives the cost estimated from the forward run, assuming the same 8 chains and the same latency of 16 s per epoch. A run at $K = 500$ would take about 2.2 h at that latency, above the limit of 2 h per `Session` set in the script.

D.3 Open choices

[**TODO:** Decidir antes de rodar: (1) *mitigação:* o padrão do script é `suppress` (TREX, desacoplamento dinâmico e twirling), mas a rodada do problema direto foi sem mitigação e o emulador não reproduz a mitigação do runtime; para comparar, use `none`; (2) *QPU:* `ibm_berlin` (compara com o direto e com $K = 110$) ou `ibm_aachen` (compara com $K = 500$); (3) *seed:* fixá-la antes de ver o QPU; se for a melhor de um lote no emulador, dizer

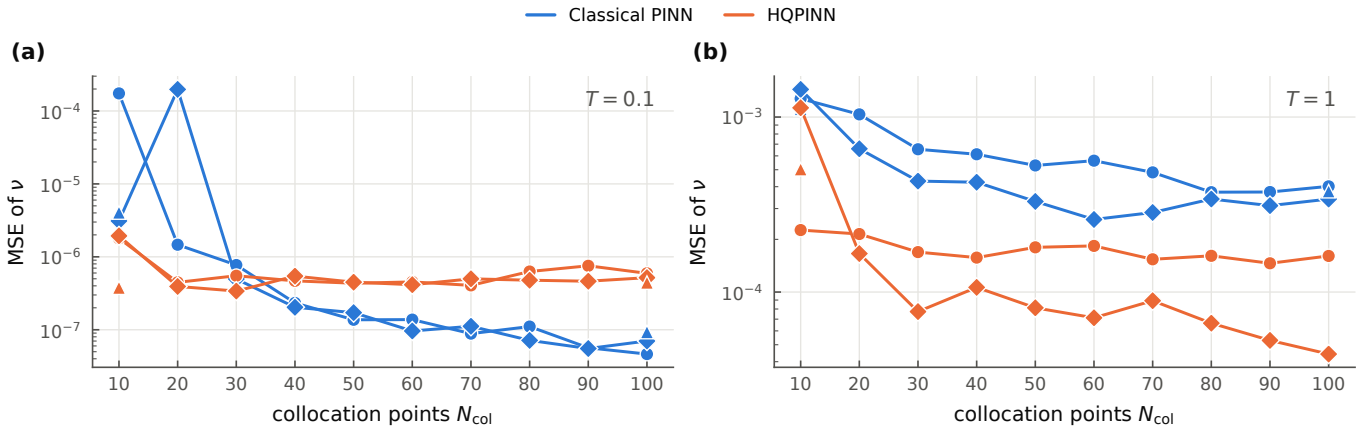


FIG. 8. Older long-budget sweeps of the inverse problem (another protocol for each model; see text): mean squared error of the learned viscosity, the mean of $(\hat{\nu} - \nu_{\mathrm{true}})^2$ over the seeds, versus the number of collocation points, for (a) $T = 0.1$ and (b) $T = 1$. Blue, classical PINN (50 seeds); orange, older noiseless HQPINN (10 seeds). Markers: circles $n = 3$, diamonds $n = 7$, triangles $n = 10$, which exists only at $N_{\mathrm{col}} = 10$ and 100.

TABLE XI. Estimated cost of an inverse run on `ibm_berlin` ($n = 10$, 8 chains, $K = 110$, $S = 1024$). These are estimates, not measurements: 128 calibration sets, $\lceil$points/8$\rceil$ sets per training job, 100 validation sets.

	$T = 0.1$	$T = 1$
jobs	112	112
points per training job	10	20
parameter sets	448	558
shots	4.6×10^5	5.7×10^5
wall-clock time	~ 1 h	~ 1 h

no texto; (4) *estatística:* uma rodada só não se compara com o emulador em ν, porque o ν final de uma rodada oscila muito; ao menos algumas seeds seriam necessárias, sobretudo em $T = 1$.]

[1] M. Raissi, P. Perdikaris, and G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. **378**, 686 (2019).

[2] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, Physics-informed machine learning, Nat. Rev. Phys. **3**, 422 (2021).

[3] A. S. Krishnapriyan, A. Gholami, S. Zhe, R. M. Kirby, and M. W. Mahoney, Characterizing possible failure modes in physics-informed neural networks, in *Advances in Neural Information Processing Systems*, Vol. 34 (2021) pp. 26548–26560.

[4] S. Wang, Y. Teng, and P. Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, SIAM J. Sci. Comput. **43**, A3055 (2021).

[5] S. Wang, X. Yu, and P. Perdikaris, When and why PINNs fail to train: A neural tangent kernel perspective, J. Comput. Phys. **449**, 110768 (2022).

[6] M. Lubasch, J. Joo, P. Moinier, M. Kiffner, and D. Jaksch, Variational quantum algorithms for nonlinear problems, Phys. Rev. A **101**, 010301(R) (2020).

[7] O. Kyriienko, A. E. Paine, and V. E. Elfving, Solving nonlinear differential equations with differentiable quantum circuits, Phys. Rev. A **103**, 052416 (2021).

[8] A. E. Paine, V. E. Elfving, and O. Kyriienko, Quantum kernel methods for solving regression problems and differential equations, Phys. Rev. A **107**, 032428 (2023).

[9] A. Sedykh, M. Podapaka, A. Sagingalieva, K. Pinto, M. Pflitsch, and A. Melnikov, Hybrid quantum physics-informed neural networks for simulating computational fluid dynamics in complex shapes, Mach. Learn.: Sci. Technol. **5**, 025045 (2024).

[10] S. Berger, N. Hosters, and M. Möller, Trainable embedding quantum physics informed neural networks for solving nonlinear PDEs, Sci. Rep. **15**, 18823 (2025).

[11] A. Farea, S. Khan, and M. S. Celebi, QCPINN: Quantum-classical physics-informed neural networks for solving PDEs, Mach. Learn.: Sci. Technol. 10.1088/2632-2153/ae1c91 (2025), arXiv:2503.16678. VERIFY volume and article number.

[12] S. Lantigua, G. Giraldi, and R. Portugal, Classical-quantum hybrid architecture for physics-informed neural networks, Phys. Rev. A **113**, 042446 (2026).

[13] K. Zabihi, H. Montazeri, and A. S. J. Suiker, Hybrid quantum-classical physics-informed neural networks for

solving nonlinear PDEs: when and where hybridization is effective? (2026), vERIFY author list, arXiv:2606.04679.
[14] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, Barren plateaus in quantum neural network training landscapes, Nat. Commun. **9**, 4812 (2018).
[15] M. Cerezo, A. Sone, T. Volkoff, L. Cincio, and P. J. Coles, Cost function dependent barren plateaus in shallow parametrized quantum circuits, Nat. Commun. **12**, 1791 (2021).
[16] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii, Quantum circuit learning, Phys. Rev. A **98**, 032309 (2018).
[17] M. Schuld, V. Bergholm, C. Gogolin, J. Izaac, and N. Killoran, Evaluating analytic gradients on quantum hardware, Phys. Rev. A **99**, 032331 (2019).
[18] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, Supervised learning with quantum-enhanced feature spaces, Nature **567**, 209 (2019).
[19] K. Fujii and K. Nakajima, Harnessing disordered-ensemble quantum dynamics for machine learning, Phys. Rev. Appl. **8**, 024030 (2017).
[20] P. Mujal, R. Martínez-Peña, J. Nokkala, J. García-Beni, G. L. Giorgi, M. C. Soriano, and R. Zambrini, Opportunities in quantum reservoir computing and extreme learning machines, Adv. Quantum Technol. **4**, 2100027 (2021).
[21] M. Henderson, S. Shakya, S. Pradhan, and T. Cook, Quanvolutional neural networks: powering image recognition with quantum circuits, Quantum Mach. Intell. **2**, 2 (2020).
[22] L. G. Wright, T. Onodera, M. M. Stein, T. Wang, D. T. Schachter, Z. Hu, and P. L. McMahon, Deep physical neural networks trained with backpropagation, Nature **601**, 549 (2022).
[23] G. H. Golub and V. Pereyra, The differentiation of pseudo-inverses and nonlinear least squares problems whose variables separate, SIAM J. Numer. Anal. **10**, 413 (1973).
[24] F. J. Schreiber, J. Eisert, and J. J. Meyer, Classical surrogates for quantum learning models, Phys. Rev. Lett. **131**, 100803 (2023).
[25] C. Basdevant, M. Deville, P. Haldenwang, J. M. Lacroix, J. Ouazzani, R. Peyret, P. Orlandi, and A. T. Patera, Spectral and finite difference solutions of the Burgers equation, Comput. Fluids **14**, 23 (1986).
[26] I. E. Lagaris, A. Likas, and D. I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, IEEE Trans. Neural Netw. **9**, 987 (1998).
[27] Y. Bengio, N. Léonard, and A. Courville, Estimating or propagating gradients through stochastic neurons for conditional computation (2013), arXiv:1308.3432.
[28] I. Loshchilov and F. Hutter, Decoupled weight decay regularization, in *International Conference on Learning Representations* (2019).
[29] D. C. Liu and J. Nocedal, On the limited memory BFGS method for large scale optimization, Math. Program. **45**, 503 (1989).
[30] M. Schuld, R. Sweke, and J. J. Meyer, Effect of data encoding on the expressive power of variational quantum-machine-learning models, Phys. Rev. A **103**, 032430 (2021).
[31] M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, and R. Ng, Fourier features let networks learn high frequency functions in low dimensional domains, in *Advances in Neural Information Processing Systems*, Vol. 33 (2020) pp. 7537–7547.
[32] A. Rahimi and B. Recht, Random features for large-scale kernel machines, in *Advances in Neural Information Processing Systems*, Vol. 20 (2007).
[33] D. Wierichs, J. Izaac, C. Wang, and C. Y.-Y. Lin, General parameter-shift rules for quantum gradients, Quantum **6**, 677 (2022).
[34] J. C. Spall, Multivariate stochastic approximation using a simultaneous perturbation gradient approximation, IEEE Trans. Autom. Control **37**, 332 (1992).
[35] A. Kandala, A. Mezzacapo, K. Temme, M. Takita, M. Brink, J. M. Chow, and J. M. Gambetta, Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets, Nature **549**, 242 (2017).
[36] K. J. Sung, J. Yao, M. P. Harrigan, N. C. Rubin, Z. Jiang, L. Lin, R. Babbush, and J. R. McClean, Using models to improve optimizers for variational quantum algorithms, Quantum Sci. Technol. **5**, 044008 (2020).
[37] E. van den Berg, Z. K. Minev, and K. Temme, Model-free readout-error mitigation for quantum expectation values, Phys. Rev. A **105**, 032620 (2022).
[38] K. Temme, S. Bravyi, and J. M. Gambetta, Error mitigation for short-depth quantum circuits, Phys. Rev. Lett. **119**, 180509 (2017).
[39] K. M. Nakanishi, K. Fujii, and S. Todo, Sequential minimal optimization for quantum-classical hybrid algorithms, Phys. Rev. Res. **2**, 043158 (2020).
[40] M. Ostaszewski, E. Grant, and M. Benedetti, Structure optimization for parameterized quantum circuits, Quantum **5**, 391 (2021).
[41] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, Quantum computing with Qiskit (2024), arXiv:2405.08810.
[42] J. D. Cole, On a quasi-linear parabolic equation occurring in aerodynamics, Q. Appl. Math. **9**, 225 (1951).
[43] E. Hopf, The partial differential equation $u_t + uu_x = \mu u_{xx}$, Commun. Pure Appl. Math. **3**, 201 (1950).
[44] I. Loshchilov and F. Hutter, SGDR: Stochastic gradient descent with warm restarts, in *International Conference on Learning Representations (ICLR)* (2017).